

Kommander

Building Distributed Systems in **.NET**
with Embedded **Raft**



Kommander: Building Distributed Systems in .NET with Embedded Raft

**A practical guide to replicated services, partitioned
consensus, and production-grade coordination**

Andres Gutierrez

2026

Contents

1	Preface	4
1.1	Who This Book Is For	4
1.2	How to Read This Book	4
2	The Problem with One Copy	5
2.1	A Simple Service	5
2.2	The Naive Fix: Run Two Copies	6
2.3	Why “Just Copy the Data” Does Not Work	7
2.4	The Real Requirement	7
2.5	One Leader, One Order	8
2.6	Consensus	8
2.7	What Raft Provides	9
2.8	Embedded Consensus	10
2.9	What This Means for the Key/Value Service	11
2.10	Summary	12
3	Introducing Kommander	12
3.1	What Kommander Is	12
3.2	The Alternative: External Coordination	13
3.3	The Embedded Approach	14
3.4	Raft in 60 Seconds	14
3.5	What Kommander Provides vs. What You Provide	15
3.6	The Central Mental Model	16
3.7	Partitions	17
3.8	The Pluggable Architecture	17
3.9	The Entry Point: RaftManager	18
3.10	The Lifecycle	19
3.11	IRaft: The Interface	20
3.12	What This Means for the Key/Value Service	20
3.13	Summary	21
4	Your First Replicated Command	21
4.1	Setting Up the Project	22

4.2	Configuring the Node	22
4.3	Choosing the Adapters	23
4.4	Registering the Callback	23
4.5	Building the ReplicatedStore	24
4.6	Walking Through the Code	27
4.7	The Lifecycle of One Command	29
4.8	Testing It	30
4.9	What Happens When the Process Stops	30
4.10	Adding Durability: RocksDbWAL	30
4.11	Rebuilding State on Restart: OnLogRestored	31
4.12	Partition 0 vs. Partition 1	32
4.13	Summary	33
5	Running a Three-Node Cluster	33
5.1	Why Three Nodes	33
5.2	What Changes from Chapter 3	34
5.3	The Three-Node Architecture	34
5.4	The Complete Node Code	35
5.5	Walking Through the New Parts	39
5.6	Starting the Cluster	40
5.7	Leader Election	41
5.8	Writing to the Cluster	41
5.9	Reading from Any Node	42
5.10	Surviving a Follower Failure	42
5.11	Surviving a Leader Failure	42
5.12	The Quorum Boundary	43
5.13	Terms	43
5.14	Data Directory Layout	44
5.15	Summary	44
6	Leadership and Request Routing	44
6.1	Only the Leader Accepts Writes	45
6.2	Three Ways to Check Leadership	45
6.3	Finding the Leader	47
6.4	Routing Strategies	48
6.5	The Stale Leader Problem	50
6.6	The OnLeaderChanged Event	52
6.7	Timing Parameters	52
6.8	Putting It Together: The ReplicatedStore Write Handler	52
6.9	Summary	54
7	Strongly Consistent Reads	54
7.1	The Problem with Local Reads	54
7.2	Why Writes Do Not Have This Problem	55
7.3	The Read-Index Protocol	55

7.4	ConfirmLeadershipAsync	56
7.5	Adding Consistent Reads to ReplicatedStore	57
7.6	The Cost of Consistency	58
7.7	Consistent Reads on Followers: ConfirmLocalApplicationAsync	59
7.8	When Not to Use Consistent Reads	60
7.9	The Complete Stale-Read Scenario	61
7.10	Internals: How It Works	62
7.11	Summary	62
8	Replication Deep Dive	63
8.1	The Full Lifecycle	63
8.2	Step 1: Admission	64
8.3	Step 2: Propose	64
8.4	Step 3: Replicate	65
8.5	Step 4: Quorum	65
8.6	Step 5: Commit	66
8.7	Step 6: Apply	66
8.8	Step 7: Return	66
8.9	The Ambiguity Window	67
8.10	ReplicateLogs Overloads	68
8.11	Observing the Lifecycle	70
8.12	Timeline: Three Nodes Processing One Write	71
8.13	What Happens When Things Go Wrong	72
8.14	Summary	72
9	Designing Application State Machines	73
9.1	The Core Rule: Determinism	73
9.2	What Breaks Determinism	73
9.3	The Two Callbacks	77
9.4	Command Serialization	79
9.5	Command Versioning	80
9.6	Idempotency	81
9.7	Building the ReplicatedStore State Machine	82
9.8	The Boundary Diagram	84
9.9	Checklist	84
9.10	Summary	85
10	Batch Replication and Manual Commit Control	85
10.1	Batch Replication: Homogeneous	85
10.2	Batch Replication: Heterogeneous	87
10.3	Manual Commit	89
10.4	The Reserve-and-Confirm Pattern	92
10.5	Manual Commit with Heterogeneous Batches	94
10.6	What Happens When the Leader Crashes	95
10.7	Checkpoints	96

10.8	Building a Bulk Import Endpoint	96
10.9	Choosing the Right Approach	98
10.10	Summary	98
11	Why One Raft Group Is Not Enough	99
11.1	The Single-Leader Bottleneck	99
11.2	Partitioned Raft	100
11.3	Configuring Partitions	101
11.4	Partition 0: The System Partition	102
11.5	Routing Keys to Partitions	102
11.6	The Partition Map	106
11.7	Failure Isolation	107
11.8	A Note on the Word “Partition”	108
11.9	Summary	108

1 Preface

This book teaches you how to build distributed systems in .NET using Kommander, an open-source embedded Raft consensus library for C#.

Kommander is a library, not a database. It gives your application the machinery for multiple nodes to agree on ordered changes and recover after failures. You keep control of your API, your domain model, your state machine, and your persistence.

The book starts with a concrete problem: three instances of a service must apply the same changes in the same order without losing data when one node crashes. It builds from that problem through working code, distributed systems concepts, failure scenarios, and source-code walkthroughs.

By the end, you will be able to build, operate, and test a production-grade replicated service.

1.1 Who This Book Is For

You are a professional .NET developer who builds backend services. You are comfortable with C#, ASP.NET Core, asynchronous programming, and databases. You may know what replication means without knowing the details of Raft, quorum intersection, or log matching.

1.2 How to Read This Book

The book is organized in six parts. Parts I through III take you from zero to a partitioned replicated service. Part IV covers production operations. Part V explains the implementation internals. Part VI covers testing and verification.

You do not need to read the Raft paper first. The book introduces every concept before it depends on it.

2 The Problem with One Copy

You have a service. It stores state: customer records, account balances, session tokens, configuration entries. The service runs on one machine, backed by one process and one in-memory data structure. It works. Clients send requests, the service updates state, and everything is consistent because there is exactly one copy of the truth.

Then the machine crashes.

Every client that depended on that state is now staring at a connection timeout. If the state lived only in memory, it is gone. If it lived on disk, it is at least unavailable until the machine comes back, assuming it comes back at all. This is the oldest problem in distributed systems: a single point of failure.

The obvious fix is to run more than one copy. But the moment you do, a harder problem appears.

2.1 A Simple Service

Let's make this concrete. Here is a minimal ASP.NET Core key/value service:

```
using System.Collections.Concurrent;

var store = new ConcurrentDictionary<string, string>();

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapPut("/kv/{key}", (string key, HttpRequest request) =>
{
    using var reader = new StreamReader(request.Body);
    string value = reader.ReadToEndAsync().Result;
    store[key] = value;
    return Results.Ok(new { key, value });
});

app.MapGet("/kv/{key}", (string key) =>
{
    return store.TryGetValue(key, out string? value)
        ? Results.Ok(new { key, value })
        : Results.NotFound();
});

app.MapGet("/kv", () =>
{
    return Results.Ok(store.ToDictionary(x => x.Key, x => x.Value));
});

app.Run();
```

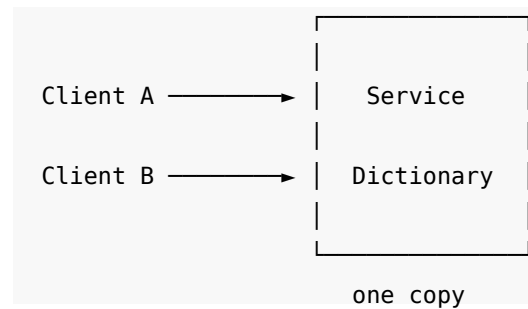
This service does exactly what it should. A client writes a value:

```
PUT /kv/customer:42 → body: "active"
```

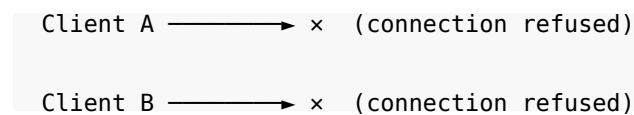
Another client reads it:

```
GET /kv/customer:42 → { "key": "customer:42", "value": "active" }
```

The dictionary is the entire state of the system. It lives in one process, on one machine. As long as that process is running, every client sees the same state.



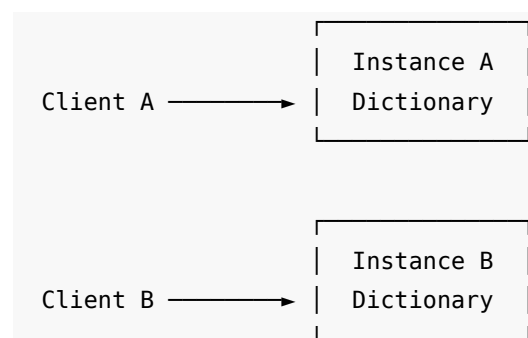
Now kill the process.



Every key is gone. Every client is blocked. The service has a single point of failure in two dimensions: it is a single point of failure for **availability** (no one can reach the data) and for **durability** (the data itself may be lost).

2.2 The Naive Fix: Run Two Copies

The instinct is to run a second instance. Maybe a load balancer distributes requests between them:



This is worse than the single-copy system. Not because it fails more often, but because it fails *silently*.

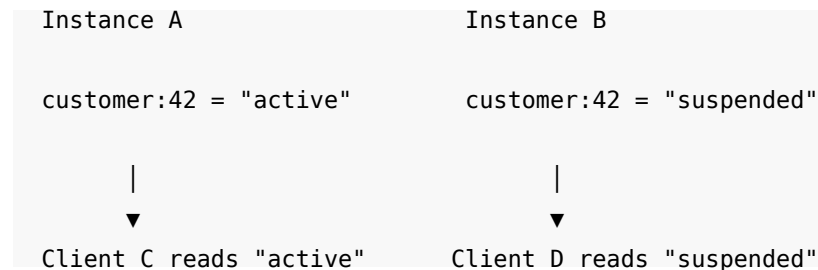
Client A writes to Instance A:

```
PUT /kv/customer:42 → "active" (on Instance A)
```

Client B writes to Instance B:

PUT /kv/customer:42 → "suspended" (on Instance B)

Now Client C reads from Instance A and sees "active". Client D reads from Instance B and sees "suspended". Neither response is wrong. Both instances faithfully served whatever was written to them. But the system as a whole is no longer consistent. Two clients asked the same question and received different answers.



This is **divergence**. The two copies of state are no longer equivalent, and no mechanism exists to reconcile them. Running two independent copies of a stateful service does not produce redundancy. It produces two independent systems that happen to have the same API.

2.3 Why “Just Copy the Data” Does Not Work

You might think: what if Instance A forwarded every write to Instance B? Then both would have the same data.

This breaks down as soon as two writes arrive at the same time.

Consider two concurrent requests:

Client A → Instance A: PUT /kv/balance "100"

Client B → Instance B: PUT /kv/balance "200"

Each instance applies its own write locally. Then each forwards the write to the other instance. Now both instances have received both writes. But in which order?

Instance A: balance = "100" → receives "200" → balance = "200"

Instance B: balance = "200" → receives "100" → balance = "100"

The final state depends on the order in which the forwarded writes arrive. And that order depends on the network, which offers no guarantees. Instance A ends up with "200" and Instance B ends up with "100". The system has diverged again, despite both instances having processed the same set of writes.

This is the **ordering problem**. Two instances cannot independently decide the order of concurrent operations and arrive at the same result. The problem is not communication. Both instances received both writes. The problem is that no one decided which write goes first.

2.4 The Real Requirement

What we actually need is a system where:

1. Multiple nodes hold copies of the same state.

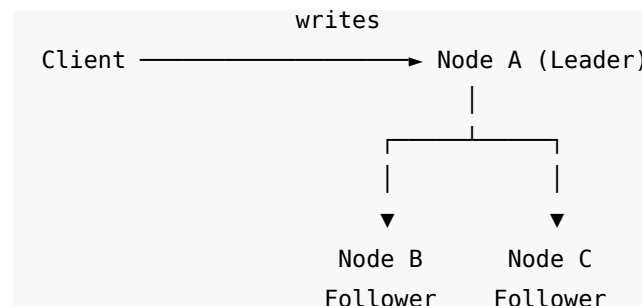
2. When a client writes to any node, every node eventually reflects that write.
3. All nodes apply writes in the **same order**.
4. If a node crashes, the remaining nodes continue to serve correct state.
5. When the crashed node recovers, it catches up to the current state.

Requirements 1 and 2 describe replication. Requirement 3 is the hard one. Requirement 4 is availability. Requirement 5 is recovery.

Requirement 3, applying writes in the same order, is what distinguishes a replicated system from a collection of independent copies. Without a shared ordering, replication produces divergence. With a shared ordering, every node is a faithful replica of a single logical state machine.

2.5 One Leader, One Order

One approach to establishing a shared order: designate one node as the **leader**. All writes go through the leader. The leader assigns a sequence number to each write and sends the writes, in order, to the other nodes (the **followers**).



This is the core idea behind a family of protocols called **consensus protocols**. The leader decides the order. The followers accept it. If the leader crashes, the remaining nodes elect a new leader and continue.

But even this simple model raises questions:

- What if a follower misses a write because of a network problem?
- What if the leader crashes after sending a write to some followers but not all?
- What if two nodes both believe they are the leader?
- What if a node restarts? How does it learn what it missed?
- How do the remaining nodes know the leader has actually crashed, rather than being temporarily slow?

These are not hypothetical edge cases. They are scenarios that will occur in any system that runs long enough on real hardware and real networks. A protocol that does not handle them will corrupt data silently.

2.6 Consensus

Consensus is the formal name for the problem of getting a group of nodes to agree on a single value (or, more usefully, on an ordered sequence of values). A consensus protocol provides guarantees:

- **Agreement:** all nodes that decide on a value decide on the same value.
- **Validity:** the decided value was actually proposed by some node.
- **Termination:** every non-faulty node eventually decides.

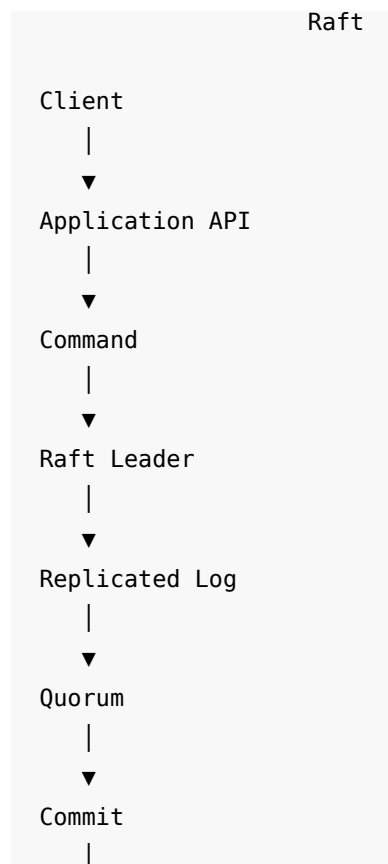
Several consensus protocols exist. Paxos is the oldest and most studied. Raft is newer, designed explicitly to be understandable. Both solve the same fundamental problem, but with different implementation approaches.

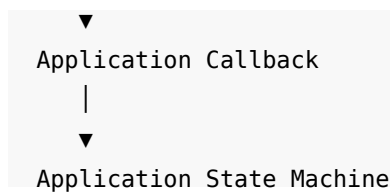
This book uses **Raft**. Not because it is better than Paxos, but because it was designed for implementors. The algorithm has a clear separation between leader election, log replication, and safety. Each piece can be understood independently.

2.7 What Raft Provides

At its core, Raft maintains a **replicated log**: an ordered sequence of entries that every node in the cluster agrees on. A leader accepts new entries, assigns them positions in the log, and replicates them to followers. An entry is **committed** once a majority of nodes have confirmed that they hold it. Once committed, the entry will not be lost even if some nodes crash.

Applications use this replicated log as a foundation. Each log entry represents a command: “set key X to value Y”, “delete key Z”, “increment counter W”. Every node processes the committed entries in the same order, and therefore every node arrives at the same state.





This separation is important. Raft replicates the *commands*. The application decides what those commands *mean*. Raft does not know anything about key/value stores, account balances, or session tokens. It guarantees that if a command is committed at position 42 in the log, then every node in the cluster will see the same command at position 42. No node will see a different command at that position.

The application takes it from there.

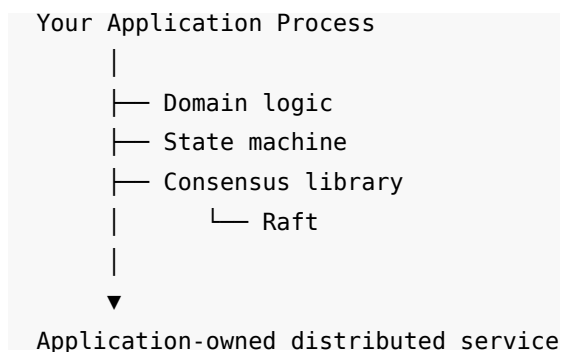
2.8 Embedded Consensus

Most developers interact with consensus indirectly. They write to etcd, or they use ZooKeeper, or they put state in a database that happens to use Raft internally. In all of these cases, the consensus protocol runs in a separate process or cluster:



This works, but it means your application depends on an external system for its most critical state. The external system must be deployed, monitored, upgraded, and secured independently. Network latency between your application and the coordination service is on the critical path of every write.

There is another approach: embed the consensus protocol directly in your application:



In this model, your service *is* the replicated system. There is no external dependency. Consensus runs in-process. Your application controls the API, the state machine, the persistence, and the routing. The consensus library provides the machinery for multiple nodes to agree on ordered commands.

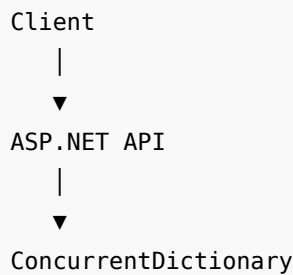
This is the model this book teaches. The library is called **Kommander**, and the next chapter introduces it.

Both approaches can coexist. **Kahuna** is an open-source distributed services platform built on top of Kommander. It provides key/value storage, distributed locking, and coordination features similar to etcd, ZooKeeper, and Consul. Kahuna uses Kommander's embedded Raft internally, but it exposes those capabilities as a standalone service that other applications can connect to. If your use case calls for an external coordination service in the .NET ecosystem, Kahuna fills that role. If you want consensus embedded directly in your own application, Kommander is the library underneath.

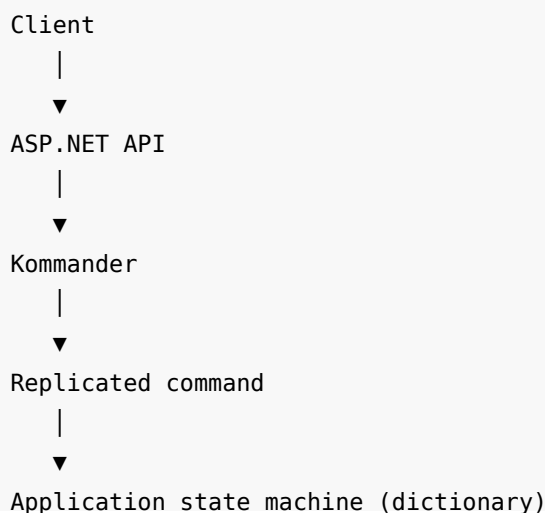
2.9 What This Means for the Key/Value Service

Let's revisit the simple service from the beginning of this chapter. By the end of Part I of this book, the in-memory dictionary will be replaced with a replicated state machine:

Before (Chapter 1):



After (Chapter 4):



The dictionary still exists, but it is no longer the source of truth. Kommander's replicated log is the source of truth. The dictionary is a projection of the committed log, built by applying each committed command, in order, to an initially empty dictionary.

When a node crashes, the remaining nodes continue to serve reads and accept writes. When the crashed node restarts, it replays the committed log entries and rebuilds the dictionary to the current state.

That is the system we will build. The next chapter introduces the tool we will use to build it.

2.10 Summary

- A single-instance stateful service is a single point of failure for both availability and durability.
- Running multiple independent copies does not produce consistency. It produces divergence.
- Replication without ordering guarantees is insufficient. Concurrent writes applied in different orders produce different state.
- Consensus protocols solve the ordering problem by ensuring all nodes agree on the same ordered sequence of commands.
- Raft is a consensus protocol designed for implementors. It separates leader election, log replication, and safety.
- Embedded consensus places the protocol inside your application process, removing the dependency on an external coordination service.
- Kommander is a .NET library that provides embedded Raft consensus. The next chapter introduces it.

3 Introducing Kommander

Chapter 1 showed why a replicated service needs consensus. Every node must apply the same writes in the same order. A leader must coordinate that order. An election must choose a new leader when the old one fails.

This chapter introduces the tool that provides these guarantees: **Kommander**, an open-source embedded Raft consensus library for C# and .NET.

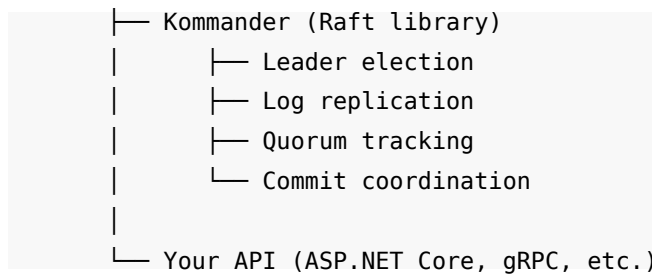
3.1 What Kommander Is

Kommander is a library, not a database. It is not a standalone server. It is not a coordination service that you deploy and connect to. It is a NuGet package that you add to your own application:

```
dotnet add package Kommander
```

After you add it, your application process contains a Raft node. That node participates in leader election, replicates commands to other nodes, and delivers committed commands to your code through a callback. Your application controls the API, the state machine, and the domain logic. Kommander provides the replication machinery.

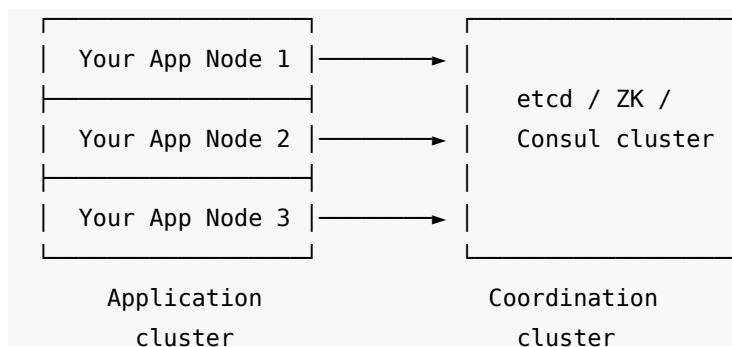
```
Your Application Process
|
├─ Your domain logic
├─ Your state machine
```



This model is called **embedded consensus**. The consensus protocol runs inside your process. There is no external service to deploy, monitor, or secure.

3.2 The Alternative: External Coordination

Most developers interact with consensus through an external coordination service. etcd, ZooKeeper, and Consul are common examples. In that model, your application connects to a separate cluster:



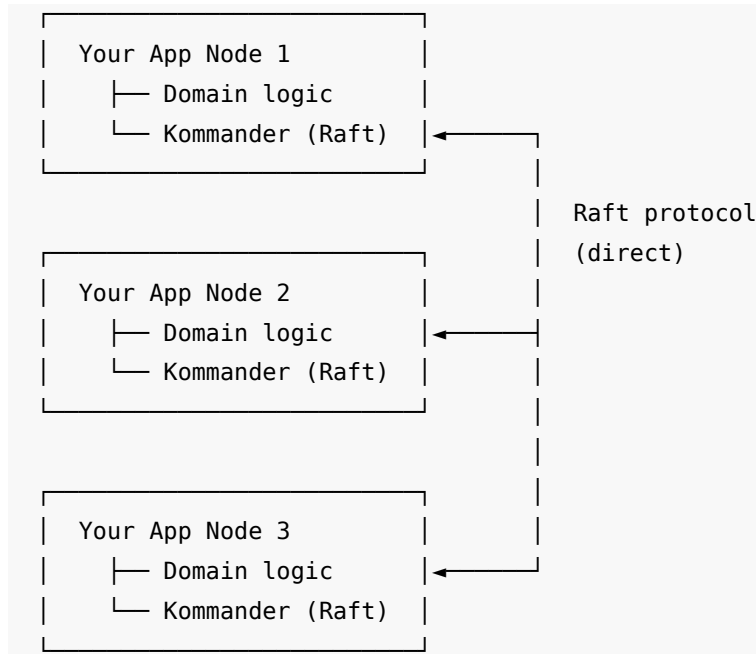
This approach works. Many production systems rely on it. In the .NET ecosystem, **Kahuna** is an open-source distributed services platform that fills this role. Kahuna provides key/value storage, distributed locking, and coordination as a standalone service, similar to etcd, ZooKeeper, and Consul. Under the hood, Kahuna uses Kommander's embedded Raft for its consensus layer.

But the external model has costs:

1. **Operational dependency.** The coordination cluster must be deployed, upgraded, and monitored independently from your application. If it goes down, your application cannot coordinate.
2. **Network latency.** Every write passes through the network to the coordination service and back. That round trip sits on the critical path.
3. **Schema mismatch.** The coordination service has its own data model (key/value, znodes, service entries). You must translate between your domain and the service's model.
4. **Capacity limits.** The coordination service has its own throughput limits. Your application must share them with all other clients of that service.

3.3 The Embedded Approach

With embedded consensus, each application node runs a Raft node inside its own process:



The Raft nodes communicate directly with each other. There is no external service. Each application process is both a Raft participant and a domain service.

This approach has its own trade-offs:

1. **Simplicity.** One deployment, not two. No separate cluster to manage.
2. **Latency.** Consensus happens in-process and over direct node-to-node links. No extra hop.
3. **Responsibility.** Your application now owns the consensus layer. You configure it, tune it, and reason about it.

Kommander is designed for the third point. It exposes the consensus primitives so that you can build on them, while it handles the protocol correctness internally.

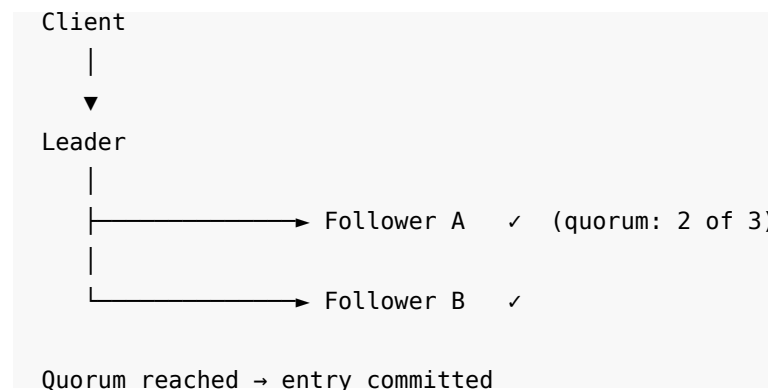
3.4 Raft in 60 Seconds

Raft is the consensus protocol that Kommander implements. Here is the essential model.

A **cluster** is a set of nodes. Each node is in one of three states for a given partition: **leader**, **follower**, or **candidate**.

- The **leader** accepts commands from clients. It assigns each command a position in an ordered log. It sends the log entries to the followers.
- The **followers** receive log entries from the leader and write them to their own logs.
- A **candidate** is a node that is running an election to become leader.

The key guarantee: a log entry is **committed** when a majority of nodes (a **quorum**) have confirmed that they hold it. Once committed, the entry will not be lost, even if some nodes crash.



When the leader crashes, the followers notice that heartbeats have stopped. One of them becomes a candidate, requests votes, and wins an election. The new leader picks up where the old one left off.

This is a simplified view. The full Raft protocol handles many edge cases: split votes, log divergence, network partitions, and more. Later chapters cover these details. For now, the important point is: Raft gives you an ordered, replicated, durable log. Your application builds its state on top of that log.

3.5 What Kommander Provides vs. What You Provide

The boundary between Kommander and your application is clear:

Kommander provides	Your application provides
Leader election	API endpoints (HTTP, gRPC, etc.)
Log replication and quorum	Domain logic and validation
Commit coordination	State machine (the thing that processes commands)
Heartbeats and failure detection	Command serialization (encoding commands as bytes)
Log persistence (WAL adapters)	Request routing (sending writes to the leader)
Cluster membership changes	Read strategy (local reads vs. confirmed reads)

Kommander does not know what your commands mean. It treats every command as a byte array with a string type label. It guarantees that if a command is committed at log position 42, every node in the cluster will see the same command at position 42. Your application decides what that command does.

Kommander's view:

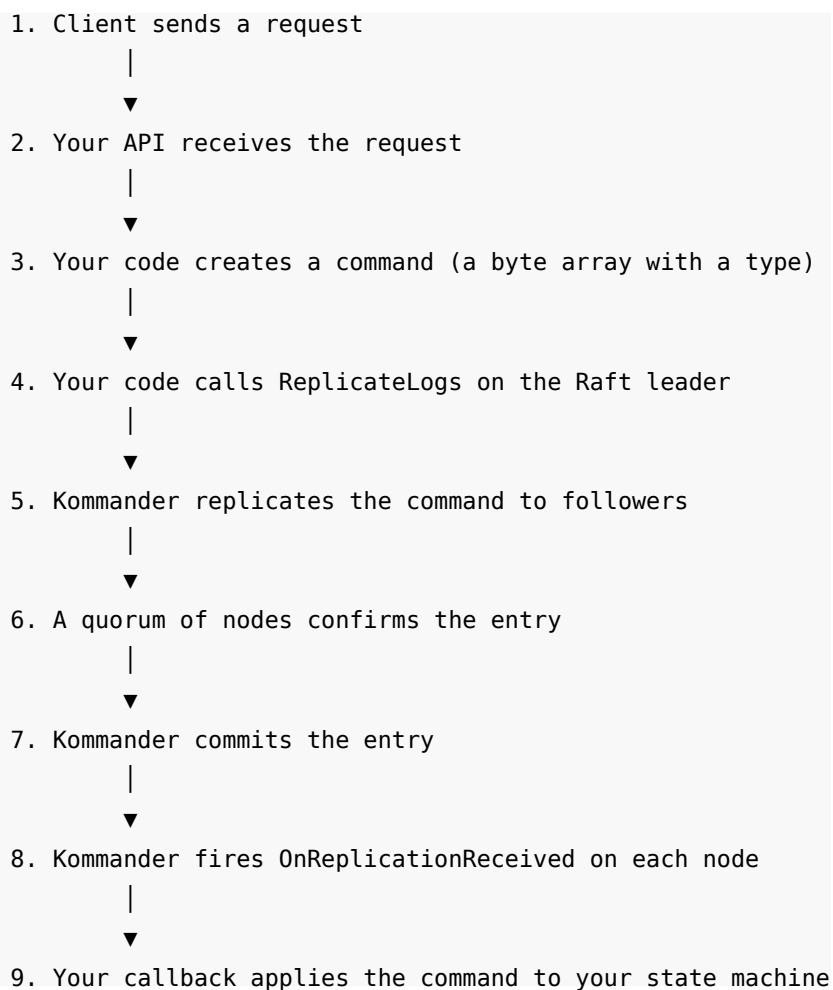
```
Log position 42:  type = "SetValue"
                  data = [0x7B, 0x22, 0x6B, ...]    (opaque bytes)
```

Your application's view:

```
Log position 42:  {"key": "customer:42", "value": "active"}
                  → dictionary["customer:42"] = "active"
```

3.6 The Central Mental Model

Every write in a Kommander-based application follows the same flow:



Steps 4 through 8 are Kommander's responsibility. Steps 1 through 3 and step 9 are yours.

This flow is the same whether you have one node or fifty. It is the same whether you use an in-memory WAL or RocksDB. It is the same whether nodes communicate over gRPC, REST, or in-memory channels.

3.7 Partitions

A single Raft group processes commands sequentially through one leader. For many applications, one group is enough. But when throughput demands grow, a single leader becomes a bottleneck.

Kommander supports **partitioned Raft**. Instead of one Raft group for all commands, you can run multiple independent groups called **partitions**. Each partition elects its own leader, maintains its own log, and operates independently.

Node A	Node B	Node C
Partition 1: Leader	Partition 1: Follower	Partition 1: Follower
Partition 2: Follower	Partition 2: Leader	Partition 2: Follower
Partition 3: Follower	Partition 3: Follower	Partition 3: Leader

A node can be leader for some partitions and follower for others. This distributes the write load across the cluster.

3.7.1 Partition 0: The System Partition

Kommander reserves **partition 0** as the **system partition**. This partition stores the cluster's own configuration: the partition map, membership roster, and other internal state. Kommander manages it automatically. Your application data uses partitions numbered 1 and above.

The `InitialPartitions` setting in `RaftConfiguration` controls how many application partitions the cluster creates at startup. The default is 1.

Partitions are a scalability feature. Part III of this book covers them in detail. For now, the examples use a single application partition (partition 1).

3.8 The Pluggable Architecture

Kommander separates the consensus core from the infrastructure it depends on. Four interfaces define that boundary:

1. **Storage (IWAL)**. The write-ahead log. Kommander ships adapters for RocksDB, SQLite, and in-memory storage. RocksDB is the production choice. In-memory is useful for tests.
2. **Communication (ICommunication)**. The network transport. Kommander ships gRPC and REST adapters, plus an in-memory adapter for tests. gRPC is the production choice.

3. **Discovery (IDiscovery).** How a node finds its peers. Static discovery uses a fixed list of endpoints. Dynamic discovery lets you update the list at runtime. Multicast discovery finds peers on the local network.
4. **Time (HybridLogicalClock).** A hybrid clock that combines wall-clock time with a logical counter. Kommander uses it for proposal tickets and causality tracking.

You choose the implementations that match your environment:

Concern	Development/Test	Production
Storage	InMemoryWAL	RocksDbWAL or SqliteWAL
Communication	InMemoryCommunication	GrpcCommunication
Discovery	StaticDiscovery	StaticDiscovery or DynamicDiscovery
Clock	HybridLogicalClock	HybridLogicalClock

This pluggability is not just a test convenience. It lets you evolve your infrastructure choices without changing your application logic. The consensus behavior stays the same regardless of which adapters you plug in.

3.9 The Entry Point: RaftManager

RaftManager is the class that brings everything together. It implements the **IRaft** interface, which defines all the operations you need: joining a cluster, checking leadership, replicating commands, and leaving the cluster.

Here is how you construct a **RaftManager**:

```
using Kommander;
using Kommander.Communication.Grpc;
using Kommander.Discovery;
using Kommander.Time;
using Kommander.WAL;
using Microsoft.Extensions.Logging;

ILoggerFactory loggerFactory = LoggerFactory.Create(
    builder => builder.AddConsole()
);

ILogger<IRaft> logger = loggerFactory.CreateLogger<IRaft>();

RaftConfiguration configuration = new()
{
    NodeName = "node-1",
    Host = "localhost",
    Port = 8001,
    InitialPartitions = 1
}
```

```
};

IRaft raft = new RaftManager(
    configuration,
    new StaticDiscovery([
        new RaftNode("localhost:8002"),
        new RaftNode("localhost:8003")
    ]),
    new RocksDbWAL(
        path: "./data",
        revision: "node-1",
        logger
    ),
    new GrpcCommunication(),
    new HybridLogicalClock(),
    logger
);
```

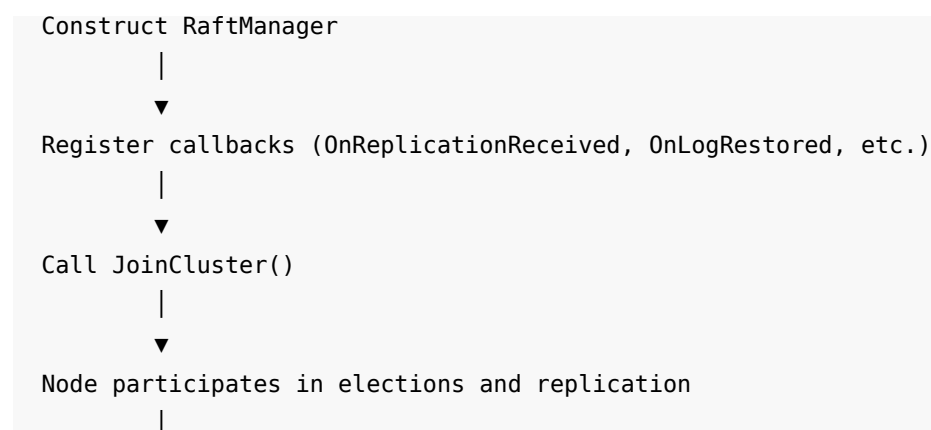
The constructor takes six arguments:

1. **RaftConfiguration** sets the node identity (name, host, port) and cluster settings (initial partitions, timeouts, intervals). Chapter 3 covers the essential properties. Appendix C is the full reference.
2. **IDiscovery** tells the node where to find its peers.
3. **IWAL** is the write-ahead log that persists Raft entries.
4. **ICommunication** is the network transport for Raft messages.
5. **HybridLogicalClock** provides timestamps for proposals.
6. **ILogger** receives diagnostic log output.

The `RaftManager` does not start any cluster activity at construction time. It only begins participating in the cluster when you call `JoinCluster`.

3.10 The Lifecycle

A Kommander node goes through a simple lifecycle:



▼
Call `LeaveCluster()` to shut down gracefully

After `JoinCluster`, the node discovers its peers, participates in leader elections, and begins accepting or forwarding commands. After `LeaveCluster`, the node stops participating and optionally disposes its resources.

The two critical callbacks are:

- **OnReplicationReceived**: fires when a committed command is ready for your application to apply. This is where you update your state machine.
- **OnLogRestored**: fires during startup for each committed entry that the WAL already contains. This is how your application rebuilds its state after a restart.

3.11 IRaft: The Interface

`IRaft` is the interface that `RaftManager` implements. It defines the full surface area that your application uses. Here are the methods and events that matter most in Part I:

Member	Purpose
<code>JoinCluster()</code>	Start the node and join the cluster
<code>LeaveCluster(dispose)</code>	Stop the node and leave the cluster
<code>AmILeader(partitionId, ct)</code>	Check if this node leads the given partition
<code>ReplicateLogs(partitionId, data, type)</code>	Propose a command for replication
<code>OnReplicationReceived</code>	Callback: a committed command is ready to apply
<code>OnLogRestored</code>	Callback: a persisted command is being replayed on startup
<code>OnLeaderChanged</code>	Callback: a new leader was elected for a partition

The remaining members of `IRaft` handle advanced features: batch replication, manual commit/rollback, partition management, read confirmation, and cluster membership. Later chapters introduce them as needed.

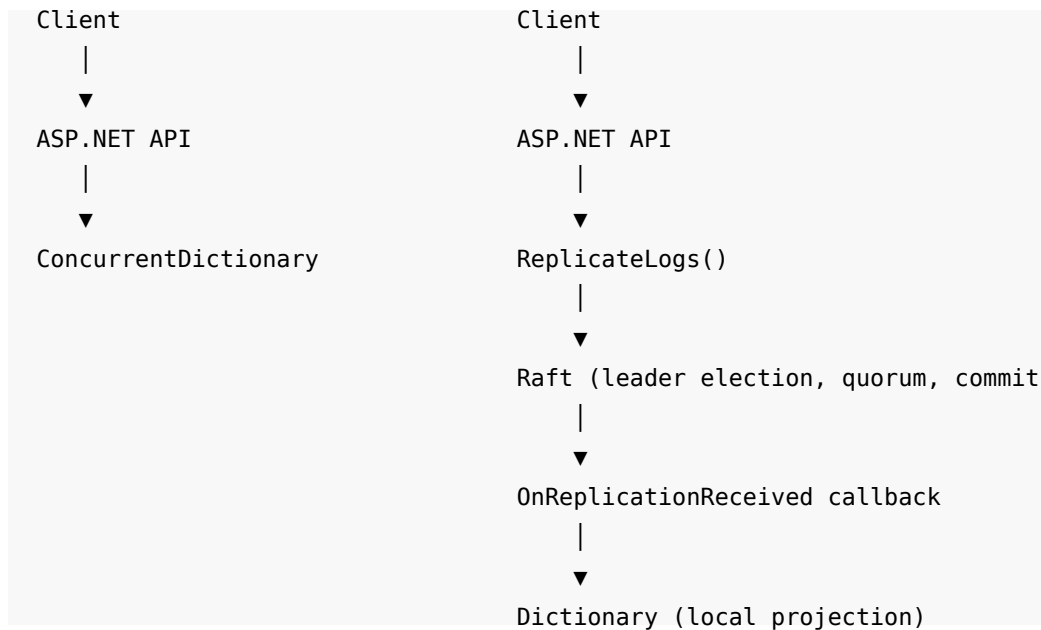
3.12 What This Means for the Key/Value Service

In Chapter 1, the key/value service used a `ConcurrentDictionary` as its only state. The dictionary was the source of truth. When the process crashed, the state was gone.

With Kommander, the dictionary still exists. But it is no longer the source of truth. The Raft log is the source of truth. The dictionary is a local projection built by applying each committed command in order.

Before (Chapter 1):

After (with Kommander):



When a node crashes, the remaining nodes continue to serve reads and accept writes. When the crashed node restarts, Kommander replays the WAL entries through the `OnLogRestored` callback. The application rebuilds the dictionary from those entries.

The next chapter builds this system step by step.

3.13 Summary

- Kommander is a .NET library that embeds Raft consensus in your application process.
- The embedded approach removes the dependency on an external coordination service.
- Raft guarantees an ordered, replicated, durable log. Your application builds state on that log.
- Kommander handles leader election, log replication, quorum, and commit. Your application handles the API, domain logic, and state machine.
- `RaftManager` is the entry point. It takes a configuration, discovery, WAL, communication, clock, and logger.
- Four pluggable interfaces (`IWAL`, `ICommunication`, `IDiscovery`, `HybridLogicalClock`) let you choose the right implementation for your environment.
- Partition 0 is the system partition. Application data uses partitions numbered 1 and above.
- The next chapter creates a single-node instance and replicates the first command.

4 Your First Replicated Command

This chapter builds a working replicated service. By the end, you will have an ASP.NET Core API that proposes commands through Kommander and applies them in a callback. The service runs on a single node. Even with one node, every step of the Raft lifecycle

executes: propose, append to the WAL, reach quorum (trivially, a quorum of one), commit, and deliver the committed entry to your application.

4.1 Setting Up the Project

Create a new ASP.NET Core web project and add the Kommander package:

```
dotnet new web -n ReplicatedStore
cd ReplicatedStore
dotnet add package Kommander
```

Kommander requires a logger, so the default `WebApplication.CreateBuilder` provides everything you need.

4.2 Configuring the Node

Every Kommander node needs a `RaftConfiguration`. The configuration tells the node its own identity (name, host, port) and how many application partitions to create at startup.

```
RaftConfiguration configuration = new()
{
    NodeName = "node-1",
    Host = "localhost",
    Port = 8001,
    InitialPartitions = 1
};
```

Five properties matter for this chapter:

Property	Purpose
<code>NodeName</code>	A human-readable name for log output. If you do not set <code>NodeId</code> , Kommander derives a numeric id from this name.
<code>Host</code>	The hostname or IP address that peer nodes use to reach this node.
<code>Port</code>	The TCP port for Raft communication.
<code>InitialPartitions</code>	The number of application partitions the cluster creates on first startup. Default is 1.
<code>NodeId</code>	A numeric identity, unique in the cluster. Optional when <code>NodeName</code> is set.

`RaftConfiguration` has many more properties for timeouts, WAL tuning, failure detection, and security. Appendix C covers them all. The defaults are safe for getting started.

4.3 Choosing the Adapters

Chapter 2 described Kommander’s four pluggable interfaces. For a single-node development setup, the simplest combination is:

Concern	Adapter	Why
Storage	InMemoryWAL	No disk setup. State lives only in memory.
Communication	InMemoryCommunication	No network setup. Only works for a single process.
Discovery	StaticDiscovery	A fixed list of peer endpoints. Empty for a single node.
Clock	HybridLogicalClock	The same clock used in production.

```
using Kommander;
using Kommander.Communication.Memory;
using Kommander.Discovery;
using Kommander.Time;
using Kommander.WAL;

ILoggerFactory loggerFactory = LoggerFactory.Create(
    builder => builder.AddConsole()
);

ILogger<IRaft> logger = loggerFactory.CreateLogger<IRaft>();

IRaft raft = new RaftManager(
    configuration,
    new StaticDiscovery([]),
    new InMemoryWAL(logger),
    new InMemoryCommunication(),
    new HybridLogicalClock(),
    logger
);
```

Note the empty list in `StaticDiscovery`. A single-node cluster has no peers. Kommander still runs the full Raft lifecycle. The node holds an election, votes for itself, and wins with a quorum of one.

4.4 Registering the Callback

Before you call `JoinCluster`, register the callback that receives committed commands. The `OnReplicationReceived` event fires each time Kommander commits an entry on a partition.

The callback receives two arguments:

1. The **partition id** (an `int`). This tells you which partition the entry belongs to.
2. A **RaftLog** object. This contains the entry's metadata and payload.

The **RaftLog** fields you will use in this chapter:

Field	Type	Meaning
Id	long	The log position (monotonically increasing within a partition).
LogType	string	The application-defined type label you passed to <code>ReplicateLogs</code> .
LogData	byte[]	The application-defined payload you passed to <code>ReplicateLogs</code> .
Type	RaftLogType	The log entry state: <code>Proposed</code> , <code>Committed</code> , <code>CommittedCheckpoint</code> , etc.
Term	long	The Raft term in which this entry was proposed.

The callback must return `Task<bool>`. Return `true` to indicate that the application processed the entry. Return `false` to signal a problem (Kommander logs a warning but does not retry).

4.5 Building the ReplicatedStore

Here is the complete single-node replicated key/value store. Read through it first. The sections that follow explain each part.

```
using System.Collections.Concurrent;
using System.Text;
using System.Text.Json;
using Kommander;
using Kommander.Communication.Memory;
using Kommander.Discovery;
using Kommander.Time;
using Kommander.WAL;

var builder = WebApplication.CreateBuilder(args);

// --- State machine: a plain dictionary ---
var store = new ConcurrentDictionary<string, string>();
```

```

// --- Kommander setup ---
ILogger<IRaft> raftLogger = builder.Services
    .BuildServiceProvider()
    .GetRequiredService<ILoggerFactory>()
    .CreateLogger<IRaft>();

RaftConfiguration configuration = new()
{
    NodeName = "node-1",
    Host = "localhost",
    Port = 8001,
    InitialPartitions = 1
};

IRaft raft = new RaftManager(
    configuration,
    new StaticDiscovery([]),
    new InMemoryWAL(raftLogger),
    new InMemoryCommunication(),
    new HybridLogicalClock(),
    raftLogger
);

// --- Callback: apply committed commands to the dictionary ---
raft.OnReplicationReceived += (partitionId, log) =>
{
    if (log.LogType == "SetValue")
    {
        var command = JsonSerializer.Deserialize<SetValueCommand>(
            log.LogData ?? []
        );

        if (command is not null)
            store[command.Key] = command.Value;
    }

    return Task.FromResult(true);
};

// --- Join the cluster ---
await raft.JoinCluster();

// --- Wait for this node to become leader ---
int partitionId = 1;

```

```

using var leaderTimeout = new CancellationTokencSource(TimeSpan.FromSeconds(10));
await raft.WaitForLeader(partitionId, leaderTimeout.Token);

// --- API endpoints ---
var app = builder.Build();

app.MapPut("/store/{key}", async (string key, HttpRequest request) =>
{
    using var reader = new StreamReader(request.Body);
    string value = await reader.ReadToEndAsync();

    byte[] payload = JsonSerializer.SerializeToUtf8Bytes(
        new SetValueCommand(key, value)
    );

    var result = await raft.ReplicateLogs(partitionId, "SetValue", payload);

    return result.Success
        ? Results.Ok(new { key, value, logIndex = result.LogIndex })
        : Results.StatusCode(503, new { error = result.Status.ToString() });
});

app.MapGet("/store/{key}", (string key) =>
{
    return store.TryGetValue(key, out string? value)
        ? Results.Ok(new { key, value })
        : Results.NotFound();
});

app.MapGet("/store", () =>
{
    return Results.Ok(store.ToDictionary(x => x.Key, x => x.Value));
});

app.Lifetime.ApplicationStopping.Register(() =>
{
    raft.LeaveCluster(dispose: true).Wait();
});

app.Run();

// --- Command record ---
record SetValueCommand(string Key, string Value);

```

4.6 Walking Through the Code

4.6.1 The State Machine

```
var store = new ConcurrentDictionary<string, string>();
```

The dictionary is the application's state machine. It is the same data structure from Chapter 1. The difference is that writes no longer go directly to the dictionary. Writes go through Kommander first.

4.6.2 The Write Path

When a client sends `PUT /store/customer:42` with body `"active"`, the handler does the following:

1. Serializes the key and value into a JSON byte array.
2. Calls `ReplicateLogs` with partition id 1, type `"SetValue"`, and the serialized payload.
3. Kommander appends the entry to the WAL, replicates it (trivially, to itself), and commits it.
4. Returns the result to the client.

```
var result = await raft.ReplicateLogs(partitionId, "SetValue", payload);
```

`ReplicateLogs` is an async method. It returns a `RaftReplicationResult` with four properties:

Property	Type	Meaning
Success	bool	true if the command was committed.
Status	RaftOperationStatus	The specific outcome: Success, NodeIsNotLeader, ProposalTimeout, etc.
LogIndex	long	The log position assigned to this entry.
TicketId	HLCTimestamp	The hybrid logical clock timestamp that identifies this proposal.

The `autoCommit` parameter defaults to `true`. This means Kommander commits the entry automatically after a quorum confirms it. Chapter 10 covers manual commit and rollback.

4.6.3 The Commit Callback

After `ReplicateLogs` commits the entry, Kommander fires `OnReplicationReceived` on every node that holds the partition. On a single-node cluster, that means this node only.

```
raft.OnReplicationReceived += (partitionId, log) =>
{
    if (log.LogType == "SetValue")
```

```

    {
        var command = JsonSerializer.Deserialize<SetValueCommand>(
            log.LogData ?? []
        );

        if (command is not null)
            store[command.Key] = command.Value;
    }

    return Task.FromResult(true);
};

```

The callback checks `log.LogType` to determine the command type. This is the same string you passed to `ReplicateLogs`. The callback deserializes `log.LogData`, extracts the key and value, and applies them to the dictionary.

This is the core pattern for every Kommander application: the write path proposes commands as opaque byte arrays, and the callback interprets them.

4.6.4 The Read Path

```

app.MapGet("/store/{key}", (string key) =>
{
    return store.TryGetValue(key, out string? value)
        ? Results.Ok(new { key, value })
        : Results.NotFound();
});

```

Reads go directly to the dictionary. There is no Raft involvement in the read path. On a single-node cluster, this is safe because the only node is always the leader and always has the latest state.

On a multi-node cluster, reading from a follower can return stale data. Chapter 6 covers strongly consistent reads.

4.6.5 Joining and Leaving

```
await raft.JoinCluster();
```

`JoinCluster` starts the node. It contacts the discovery service, discovers peers (none, in this case), and begins the Raft lifecycle. The node runs an election, votes for itself, and becomes leader.

```
await raft.WaitForLeader(partitionId, leaderTimeout.Token);
```

`WaitForLeader` blocks until a leader is elected for the given partition. On a single node, this happens almost instantly. The method returns the leader's endpoint as a string. In a multi-node cluster, you use this to know which node to send writes to.

```

app.Lifetime.ApplicationStopping.Register(() =>
{

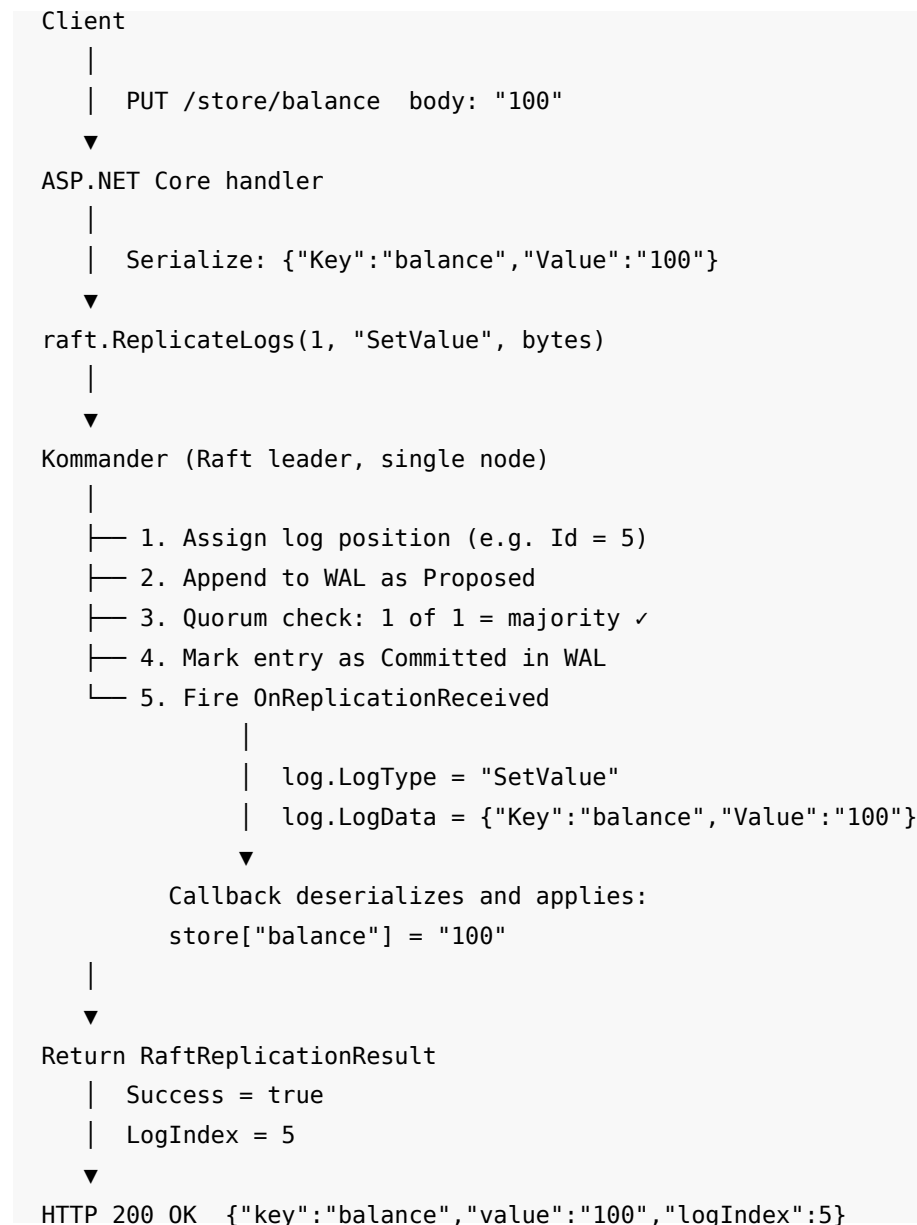
```

```
raft.LeaveCluster(dispose: true).Wait();
});
```

LeaveCluster stops the node. The `dispose: true` parameter also disposes the `RaftManager` and its resources. The shutdown hook ensures the node leaves cleanly when the process stops.

4.7 The Lifecycle of One Command

Here is the full sequence when a client writes `PUT /store/balance "100"`:



The entire sequence is synchronous from the client's perspective. `ReplicateLogs` does not return until the entry is committed. When the client receives a successful response, the command is in the log and applied to the state machine.

4.8 Testing It

Start the application:

```
dotnet run
```

Write a value:

```
curl -X PUT http://localhost:5000/store/customer:42 \  
-H "Content-Type: text/plain" \  
-d "active"
```

Read it back:

```
curl http://localhost:5000/store/customer:42
```

List all entries:

```
curl http://localhost:5000/store
```

4.9 What Happens When the Process Stops

Stop the process with Ctrl+C. Start it again with `dotnet run`.

The dictionary is empty. Every key is gone.

This is expected. `InMemoryWAL` stores the log in memory. When the process exits, the log is lost. The dictionary is a projection of the log. If the log is gone, the dictionary starts empty.

This is the same problem from Chapter 1. But now the fix is different. In Chapter 1, there was no log. The dictionary was the only state. In this architecture, the dictionary is derived from the log. If the log survives the restart, the dictionary can be rebuilt.

4.10 Adding Durability: RocksDbWAL

Replace the `InMemoryWAL` with a `RocksDbWAL`:

```
using Kommander.WAL;  
  
IRaft raft = new RaftManager(  
    configuration,  
    new StaticDiscovery([]),  
    new RocksDbWAL(  
        path: "./raft-data",  
        revision: "node-1",  
        raftLogger  
    ),  
    new InMemoryCommunication(),  
    new HybridLogicalClock(),  
    raftLogger  
);
```

`RocksDbWAL` takes three required arguments:

1. **path**: the directory where RocksDB stores its files.

2. **revision**: a subdirectory name that identifies this node's data. Use a value unique to this node, such as the node name.
3. **logger**: the same logger you pass to `RaftManager`.

An optional fourth argument, `syncWrites`, defaults to `true`. This means every WAL write is flushed to disk before Kommander considers it persisted. Set it to `false` only in development or testing where durability is not important.

4.11 Rebuilding State on Restart: `OnLogRestored`

When the node starts with a durable WAL, Kommander reads all committed entries from disk and fires the `OnLogRestored` callback for each one. This is how your application rebuilds its state machine after a restart.

Register `OnLogRestored` before calling `JoinCluster`:

```
raft.OnLogRestored += (partitionId, log) =>
{
    if (log.LogType == "SetValue")
    {
        var command = JsonSerializer.Deserialize<SetValueCommand>(
            log.LogData ?? []
        );

        if (command is not null)
            store[command.Key] = command.Value;
    }

    return Task.FromResult(true);
};
```

The code is identical to the `OnReplicationReceived` callback. The same logic that applies new commands also rebuilds state from persisted commands. In many applications, you can extract a shared method:

```
Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
    if (log.LogType == "SetValue")
    {
        var command = JsonSerializer.Deserialize<SetValueCommand>(
            log.LogData ?? []
        );

        if (command is not null)
            store[command.Key] = command.Value;
    }

    return Task.FromResult(true);
}
```

```

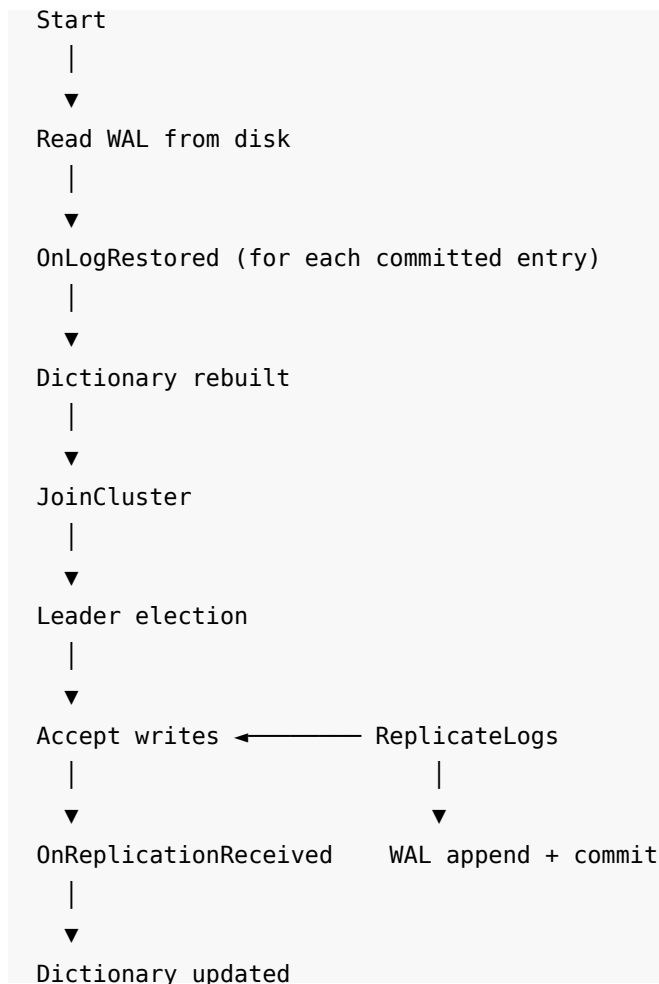
}

raft.OnReplicationReceived += ApplyCommand;
raft.OnLogRestored += ApplyCommand;

```

Now the lifecycle is complete:

1. Node starts. Kommander reads the WAL.
2. `OnLogRestored` fires for each committed entry. The dictionary is rebuilt.
3. Node joins the cluster and becomes leader.
4. New writes arrive. `ReplicateLogs` appends them to the WAL and commits them.
5. `OnReplicationReceived` fires. The dictionary is updated.
6. Node stops. `LeaveCluster` cleans up.
7. Node restarts. Go to step 1.



4.12 Partition 0 vs. Partition 1

You may have noticed that the code uses `partitionId = 1`, not `partitionId = 0`.

Partition 0 is the **system partition**. Kommander reserves it for internal configuration: the partition map, cluster membership, and other coordination state. Your application data belongs on partition 1 and above.

When `InitialPartitions = 1`, Kommander creates one application partition: partition 1. If you set `InitialPartitions = 8`, Kommander creates partitions 1 through 8. Partition 0 always exists regardless of this setting.

The `OnReplicationReceived` callback fires for all partitions, including partition 0. Kommander's internal entries on partition 0 use the reserved type `_RaftSystem`. Your callback should either check for your own types or ignore entries with `LogType` starting with `_Raft`.

4.13 Summary

- `RaftManager` is constructed with a configuration, discovery, WAL, communication, clock, and logger.
- `JoinCluster` starts the node. `LeaveCluster` stops it.
- `ReplicateLogs` proposes a command. The command is a byte array with a string type label.
- `OnReplicationReceived` fires when a command is committed and ready to apply. The application decides what the command means.
- Even a single node executes the full Raft lifecycle: propose, WAL append, quorum, commit, callback.
- `InMemoryWAL` is simple but loses state on restart. `RocksDbWAL` persists state to disk.
- `OnLogRestored` fires on startup for each persisted entry. Use it to rebuild the application state machine.
- Application data uses partition 1 and above. Partition 0 is reserved for Kommander's internal state.
- The next chapter adds two more nodes to create a real cluster.

5 Running a Three-Node Cluster

Chapter 3 built a replicated store on a single node. A single node does not tolerate failure. If that node crashes, every client is blocked. This chapter adds two more nodes. A three-node cluster tolerates one failure: any two nodes can form a quorum and continue operating while the third is down.

5.1 Why Three Nodes

Raft requires a **majority** (a quorum) of nodes to agree before an entry is committed. The quorum formula is $\text{floor}(N/2) + 1$, where N is the number of nodes in the cluster.

Nodes	Quorum	Failures tolerated
1	1	0
2	2	0
3	2	1
5	3	2
7	4	3

Two nodes are worse than one for fault tolerance. Both must agree for a commit, so a single failure still blocks the cluster. Three is the minimum for useful redundancy.

The general rule: a cluster of $2F + 1$ nodes tolerates F failures.

5.2 What Changes from Chapter 3

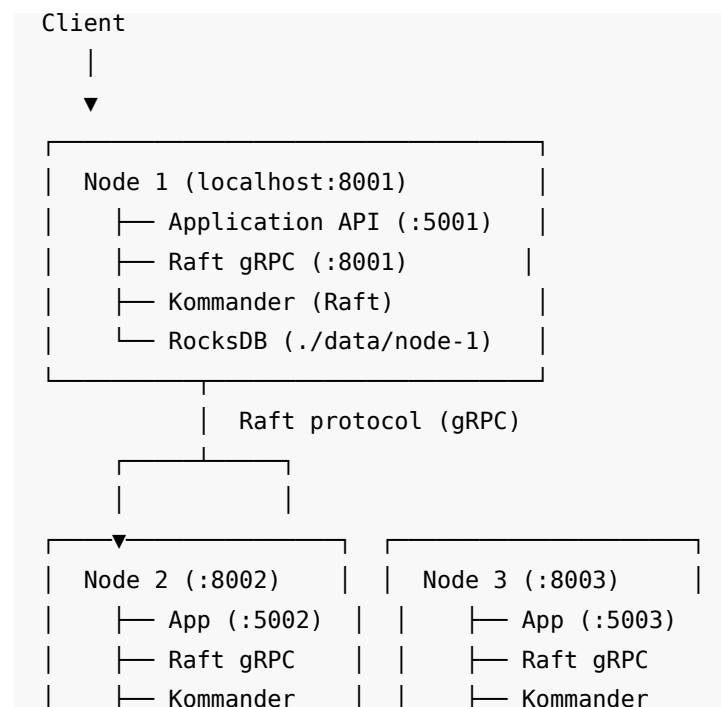
The single-node setup used `InMemoryCommunication` (no network) and `InMemoryWAL` (no disk). A three-node cluster must communicate over the network and persist state to survive restarts.

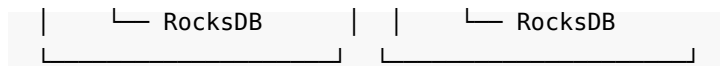
Concern	Chapter 3 (single node)	Chapter 4 (three nodes)
Communication	<code>InMemoryCommunication</code>	<code>GrpcCommunication</code>
Storage	<code>InMemoryWAL</code>	<code>RocksDbWAL</code>
Discovery	<code>StaticDiscovery([])</code>	<code>StaticDiscovery</code> with peer endpoints
Hosting	Minimal API only	ASP.NET Core with gRPC endpoints

5.3 The Three-Node Architecture

Each node runs an ASP.NET Core process that hosts two sets of endpoints:

1. **Application endpoints:** the `PUT /store/{key}` and `GET /store/{key}` routes that clients use.
2. **Raft endpoints:** gRPC services that Kommander uses for leader election, log replication, and heartbeats.





5.4 The Complete Node Code

The following program runs one node. You launch three instances of this program with different arguments for the node name, Raft port, and HTTP port. The code is longer than Chapter 3 because it includes gRPC hosting and leader-aware routing.

```

using System.Collections.Concurrent;
using System.Text;
using System.Text.Json;
using Kommander;
using Kommander.Communication.Grpc;
using Kommander.Data;
using Kommander.Discovery;
using Kommander.Time;
using Kommander.WAL;

// --- Parse command-line arguments ---
string nodeName = args.Length > 0 ? args[0] : "node-1";
int raftPort = args.Length > 1 ? int.Parse(args[1]) : 8001;
int httpPort = args.Length > 2 ? int.Parse(args[2]) : 5001;

// --- All three Raft endpoints ---
int[] allRaftPorts = [8001, 8002, 8003];

// --- ASP.NET Core builder ---
var builder = WebApplication.CreateBuilder(args);
builder.WebHost.UseUrls($"http://localhost:{httpPort}");

// Add gRPC services for Kommander
builder.Services.AddKommanderGrpc();

// --- State machine ---
var store = new ConcurrentDictionary<string, string>();

// --- Logger ---
ILogger<IRaft> raftLogger = builder.Services
    .BuildServiceProvider()
    .GetRequiredService<ILoggerFactory>()
    .CreateLogger<IRaft>();

// --- Kommander configuration ---
RaftConfiguration configuration = new()
{

```

```

    NodeName = nodeName,
    Host = "localhost",
    Port = raftPort,
    InitialPartitions = 1,
    GrpcScheme = "http://",
    HttpScheme = "http://"
};

// --- Peer discovery: list every other node ---
List<RaftNode> peers = allRaftPorts
    .Where(p => p != raftPort)
    .Select(p => new RaftNode($"localhost:{p}"))
    .ToList();

// --- Create the Raft node ---
IRaft raft = new RaftManager(
    configuration,
    new StaticDiscovery(peers),
    new RocksDbWAL(
        path: "./raft-data",
        revision: nodeName,
        raftLogger
    ),
    new GrpcCommunication(),
    new HybridLogicalClock(),
    raftLogger
);

// --- Apply committed commands ---
Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
    if (log.LogType == "SetValue")
    {
        var command = JsonSerializer.Deserialize<SetValueCommand>(
            log.LogData ?? []
        );

        if (command is not null)
            store[command.Key] = command.Value;
    }

    return Task.FromResult(true);
}

```

```

raft.OnReplicationReceived += ApplyCommand;
raft.OnLogRestored += ApplyCommand;

// --- Log leader changes ---
raft.OnLeaderChanged += (partitionId, leaderEndpoint) =>
{
    Console.WriteLine(
        $"Partition {partitionId}: leader is now {leaderEndpoint}"
    );
    return Task.FromResult(true);
};

// --- Join the cluster ---
await raft.JoinCluster();

// --- Build the application ---
var app = builder.Build();

// Map Kommander gRPC endpoints
app.MapGrpcRaftRoutes();

// --- Application endpoints ---
int partitionId = 1;

app.MapPut("/store/{key}", async (string key, HttpRequest request) =>
{
    // Check if this node is the leader
    bool isLeader = await raft.AmILeader(partitionId, default);
    if (!isLeader)
    {
        string? leaderEndpoint = raft.GetPartitionLeaderHint(partitionId);
        if (leaderEndpoint is not null)
        {
            // Find the HTTP port for the leader
            int leaderRaftPort = int.Parse(leaderEndpoint.Split(':')[1]);
            int leaderHttpPort = leaderRaftPort - 3000;
            return Results.Redirect(
                $"http://localhost:{leaderHttpPort}/store/{key}",
                permanent: false,
                preserveMethod: true
            );
        }
    }

    return Results.Json(

```



```

        new { error = "No leader available" },
        statusCode: 503
    );
}

using var reader = new StreamReader(request.Body);
string value = await reader.ReadToEndAsync();

byte[] payload = JsonSerializer.SerializeToUtf8Bytes(
    new SetValueCommand(key, value)
);

var result = await raft.ReplicateLogs(partitionId, "SetValue", payload);

return result.Success
    ? Results.Ok(new { key, value, logIndex = result.LogIndex })
    : Results.Json(
        new { error = result.Status.ToString() },
        statusCode: 503
    );
});

app.MapGet("/store/{key}", (string key) =>
{
    return store.TryGetValue(key, out string? value)
        ? Results.Ok(new { key, value })
        : Results.NotFound();
});

app.MapGet("/store", () =>
{
    return Results.Ok(store.ToDictionary(x => x.Key, x => x.Value));
});

// --- Leader info endpoint ---
app.MapGet("/leader", async () =>
{
    bool isLeader = await raft.AmILeader(partitionId, default);
    string? leaderHint = raft.GetPartitionLeaderHint(partitionId);
    return Results.Ok(new
    {
        node = nodeName,
        isLeader,
        leaderHint
    });
});

```

```

    });
});

app.Lifetime.ApplicationStopping.Register(() =>
{
    raft.LeaveCluster(dispose: true).Wait();
});

app.Run();

record SetValueCommand(string Key, string Value);

```

5.5 Walking Through the New Parts

5.5.1 gRPC Hosting

Two lines wire Kommander's gRPC services into the ASP.NET Core pipeline:

```
builder.Services.AddKommanderGrpc();
```

This registers the gRPC service with the dependency injection container. It also configures compression support for snapshot transfers.

```
app.MapGrpcRaftRoutes();
```

This maps the Raft gRPC service to the application's endpoint routing. After this call, peer nodes can reach this node's Raft endpoints over gRPC.

Both methods come from the `Kommander.Communication.Grpc` namespace.

5.5.2 Transport Scheme

```
GrpcScheme = "http://",
HttpScheme = "http://"
```

The default scheme is `https://`. For local development without TLS certificates, set both to `http://`. Chapter 20 covers transport security for production deployments.

When you use the `http://` scheme with gRPC, you also need to enable unencrypted HTTP/2. Add this environment variable before starting the process:

```
export ASPNETCORE_URLS="http://localhost:8001"
```

Or pass it on the command line. The builder's `UseUrls` call in the code above handles the HTTP endpoint. The Raft gRPC port is separate and configured through `RaftConfiguration.Port`.

5.5.3 Static Discovery with Peers

```
List<RaftNode> peers = allRaftPorts
    .Where(p => p != raftPort)
    .Select(p => new RaftNode($"localhost:{p}"))
    .ToList();
```

Each node lists every other node's Raft endpoint. Node 1 lists nodes 2 and 3. Node 2 lists nodes 1 and 3. Node 3 lists nodes 1 and 2. `StaticDiscovery` takes this list and provides it to `Kommander` when it needs to contact peers.

A `RaftNode` takes a single argument: the endpoint string in `host:port` format.

5.5.4 Leader-Aware Routing

```
bool isLeader = await raft.AmILeader(partitionId, default);
if (!isLeader)
{
    string? leaderEndpoint = raft.GetPartitionLeaderHint(partitionId);
    // ... redirect to the leader
}
```

Only the leader can accept writes through `ReplicateLogs`. If a write arrives at a follower, the handler checks `AmILeader` and redirects the client to the leader. `GetPartitionLeaderHint` returns the current leader's Raft endpoint as a best-effort hint.

This is a simple routing strategy. Chapter 5 covers more advanced approaches.

5.5.5 OnLeaderChanged

```
raft.OnLeaderChanged += (partitionId, leaderEndpoint) =>
{
    Console.WriteLine(
        $"Partition {partitionId}: leader is now {leaderEndpoint}"
    );
    return Task.FromResult(true);
};
```

This callback fires every time a new leader is elected for a partition. The two arguments are the partition id and the new leader's endpoint. You will see this fire during startup (initial election) and whenever a leader fails and a new one takes over.

5.6 Starting the Cluster

Open three terminals. In each one, start a node with different arguments:

Terminal 1 (Node 1):

```
dotnet run -- node-1 8001 5001
```

Terminal 2 (Node 2):

```
dotnet run -- node-2 8002 5002
```

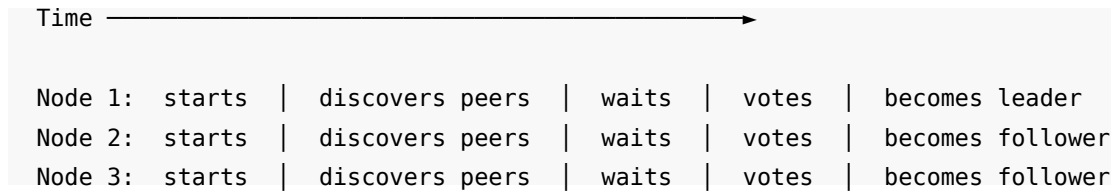
Terminal 3 (Node 3):

```
dotnet run -- node-3 8003 5003
```

Within a few seconds, the three nodes discover each other, run an election, and one becomes leader. You will see `OnLeaderChanged` messages in each terminal. All three nodes report the same leader.

5.7 Leader Election

When the three nodes start, they go through the following sequence:



Each node waits for a random duration between `StartElectionTimeout` (default: 2000 ms) and `EndElectionTimeout` (default: 4000 ms). The randomization prevents all three nodes from starting elections at the same time.

The first node whose timer expires becomes a **candidate**. It increments its **term** (a monotonically increasing number that identifies each election round) and sends vote requests to the other two nodes. If it receives votes from a majority (two of three, including itself), it becomes the leader.

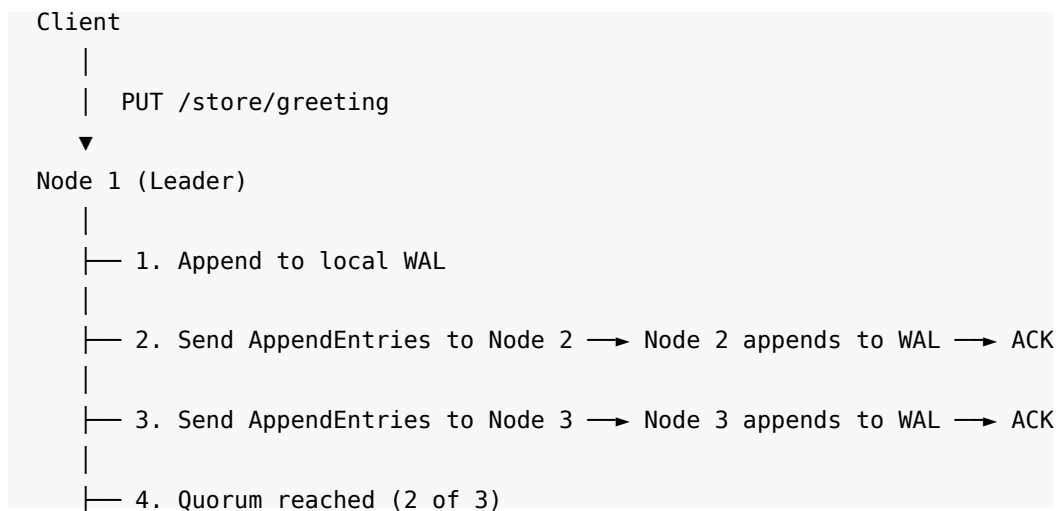
After winning, the leader sends heartbeats every `HeartbeatInterval` (default: 500 ms) to the followers. Each heartbeat resets the follower's election timer. As long as the heartbeats arrive, the followers do not start new elections.

5.8 Writing to the Cluster

Send a write to any node. If you hit the leader, the write succeeds directly. If you hit a follower, the HTTP response redirects you to the leader.

```
# Write to the leader (assume node-1 is leader)
curl -X PUT http://localhost:5001/store/greeting \
  -H "Content-Type: text/plain" \
  -d "hello from the cluster"
```

The leader proposes the command, replicates it to the two followers, and waits for a quorum. Two of three nodes must confirm the entry. After the quorum confirms, Kommander commits the entry and fires `OnReplicationReceived` on all three nodes.



```
|
|— 5. Commit
|
|— 6. OnReplicationReceived fires on all 3 nodes
```

5.9 Reading from Any Node

Read the value from any node:

```
curl http://localhost:5001/store/greeting
curl http://localhost:5002/store/greeting
curl http://localhost:5003/store/greeting
```

All three return the same value. Every node applied the committed command to its local dictionary.

Note: these reads are not strongly consistent. A follower may briefly lag behind the leader. For most applications, this is acceptable. Chapter 6 explains how to get strongly consistent reads when you need them.

5.10 Surviving a Follower Failure

Stop Node 3 with Ctrl+C. The cluster continues operating:

```
curl -X PUT http://localhost:5001/store/status \
-H "Content-Type: text/plain" \
-d "two-node quorum"
```

This write succeeds. Node 1 (leader) and Node 2 (follower) form a quorum of two. That is enough for a three-node cluster.

Restart Node 3:

```
dotnet run -- node-3 8003 5003
```

Node 3 rejoins the cluster as a follower. The leader detects that Node 3 is behind and sends the missing log entries. This is called **catch-up** or **backfill**. After catch-up completes, Node 3 has the same state as the other two nodes.

```
curl http://localhost:5003/store/status
# Returns: {"key":"status","value":"two-node quorum"}
```

The entry that was written while Node 3 was down is now available on Node 3.

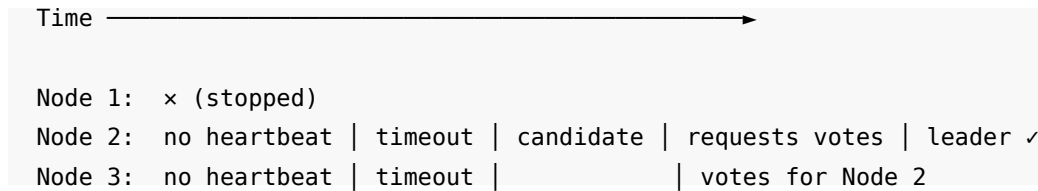
5.11 Surviving a Leader Failure

Stop the leader (Node 1):

```
Node 1 (Leader):  stopped

Node 2 (Follower): heartbeats stop arriving...
Node 3 (Follower): heartbeats stop arriving...
```

After the election timeout (2 to 4 seconds by default), one of the remaining nodes notices that heartbeats have stopped. It becomes a candidate, starts a new election, and wins:



You will see `OnLeaderChanged` messages in the terminals for Node 2 and Node 3. The new leader accepts writes:

```
# Write to the new leader
curl -X PUT http://localhost:5002/store/recovery \
  -H "Content-Type: text/plain" \
  -d "leader failover complete"
```

Restart Node 1:

```
dotnet run -- node-1 8001 5001
```

Node 1 rejoins as a follower. It catches up on the entries it missed and resumes normal operation. It does not become leader again unless the current leader fails.

5.12 The Quorum Boundary

A three-node cluster tolerates one failure. It does not tolerate two. Stop two nodes and try to write:

```
# Stop Node 2 and Node 3. Only Node 1 (if it was leader) remains.
curl -X PUT http://localhost:5001/store/test \
  -H "Content-Type: text/plain" \
  -d "should fail"
```

The write hangs or times out. One node out of three is not a quorum. Kommander cannot commit the entry because a majority cannot confirm it.

This is correct behavior. The cluster is protecting you from split-brain scenarios. If it allowed one node to commit alone, and the other two nodes were alive but partitioned away, the cluster would have two independent histories.

Start the other nodes again. The cluster recovers, the pending write (if it was still in flight) may succeed or the client can retry.

5.13 Terms

Each election increments the **term**, a monotonically increasing counter. The term serves two purposes:

1. **Ordering elections.** If a node receives a message with a higher term, it knows a newer election has occurred and steps down.
2. **Detecting stale leaders.** A leader in term 3 cannot overwrite entries from term 4.

You can observe terms in the log output. The first election might be term 1. After a leader failure and re-election, the new leader operates in term 2 (or higher, if there were split votes).

Terms are internal to Raft. Your application code does not interact with them directly. But they appear in log output and in the `RaftLog.Term` field, which is useful for debugging.

5.14 Data Directory Layout

After running the three-node cluster, the `raft-data` directory contains one subdirectory per node:

```
raft-data/  
  node-1/    ← RocksDB files for Node 1  
  node-2/    ← RocksDB files for Node 2  
  node-3/    ← RocksDB files for Node 3
```

Each node has its own RocksDB instance. The `revision` parameter in the `RocksDbWAL` constructor determines the subdirectory name. In a production deployment, each node runs on a separate machine with its own data directory.

To reset the cluster, stop all nodes and delete the `raft-data` directory:

```
rm -rf ./raft-data
```

5.15 Summary

- A three-node cluster tolerates one node failure. Any two nodes form a quorum.
- `GrpcCommunication` provides the network transport. `MapGrpcRaftRoutes()` wires the Raft gRPC service into ASP.NET Core.
- `StaticDiscovery` lists the other nodes' Raft endpoints.
- `RocksDbWAL` persists the WAL to disk. State survives restarts.
- Only the leader accepts writes through `ReplicateLogs`. Followers must redirect or forward.
- Leader election happens automatically when the current leader is unreachable. The election timeout is randomized to avoid collisions.
- `OnReplicationReceived` fires on all three nodes for every committed entry. Each node maintains its own copy of the state machine.
- `OnLeaderChanged` fires whenever a new leader is elected for a partition.
- A stopped node catches up automatically when it rejoins the cluster. The leader sends the missing entries.
- The cluster does not accept writes when fewer than a quorum of nodes are available. This prevents split-brain divergence.
- The next chapter covers leadership and request routing in more detail.

6 Leadership and Request Routing

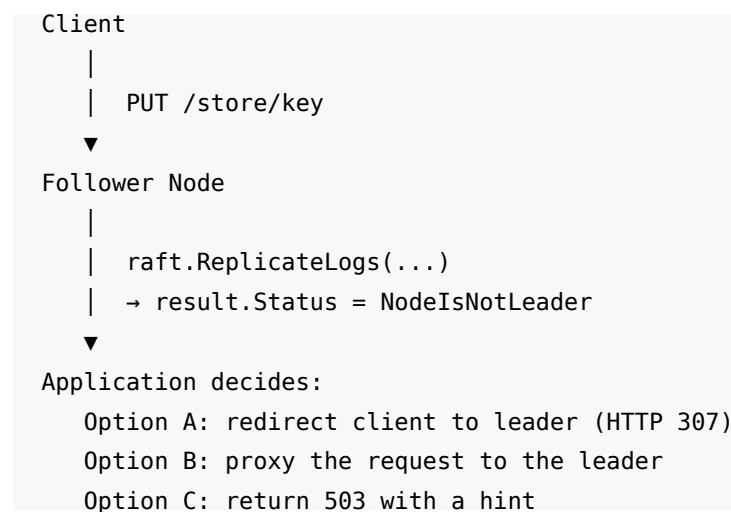
Chapter 4 built a three-node cluster and added a simple leader redirect to the write handler. This chapter goes deeper. It explains what leadership means in Raft, why local

leadership checks can be wrong, and how to build routing that handles leader changes correctly.

6.1 Only the Leader Accepts Writes

In Raft, only the leader for a partition can propose new log entries. If a follower receives a write, it cannot replicate it. The follower must either reject the request or forward it to the leader.

When you call `ReplicateLogs` on a follower in the default full-replication mode, Kommander returns a result with `Status = NodeIsNotLeader`. The entry is not proposed, not replicated, and not committed. Your application must decide what to do: redirect the client, forward the request internally, or return an error.



6.2 Three Ways to Check Leadership

Kommander provides three methods to check whether the local node leads a partition. Each has a different cost and a different guarantee.

6.2.1 AmILeaderQuick

```
bool isLeader = await raft.AmILeaderQuick(partitionId);
```

`AmILeaderQuick` reads a published in-memory field. It does not contact the partition executor. It does not contact other nodes. The cost is a single volatile read.

The guarantee: `AmILeaderQuick` returns the node's **local belief** about leadership. If the node believes it is the leader, it returns `true`. If the node knows it is not the leader, it returns `false`.

The risk: a minority-partitioned leader still believes it leads. `AmILeaderQuick` returns `true` on that node even though it cannot form a quorum. The node lost contact with the majority, but it has not yet received a higher-term message that would cause it to step down.

Use `AmILeaderQuick` for fast pre-checks where a false positive is acceptable. For example, you might use it to decide which node should run a periodic background task. If two nodes both run the task briefly during a leader transition, the system tolerates that.

6.2.2 AmILeader

```
bool isLeader = await raft.AmILeader(partitionId, cancellationToken);
```

`AmILeader` also reads published local state, but it checks the partition's role snapshot in addition to the leader endpoint. It is still a local check with no network round-trip. The cost is slightly higher than `AmILeaderQuick`, but still cheap.

The guarantee is the same: local belief. A minority-partitioned leader returns `true` from `AmILeader` just as it does from `AmILeaderQuick`. Neither method contacts other nodes to confirm.

Use `AmILeader` when you want the most current local state. It is the standard check for the write path, as shown in Chapter 4.

6.2.3 ConfirmLeadershipAsync

```
bool confirmed = await raft.ConfirmLeadershipAsync(  
    partitionId,  
    cancellationToken  
);
```

`ConfirmLeadershipAsync` contacts a quorum of nodes to confirm that this node is still the leader. It implements the **read-index protocol** from the Raft dissertation (section 6.4). This is the only leadership check that detects a minority-partitioned leader.

The guarantee: if `ConfirmLeadershipAsync` returns `true`, a majority of nodes have confirmed that this node leads the partition in the current term, and all committed entries up to that point have been applied locally. A local read after a `true` result is **linearizable**.

The cost: one quorum round-trip. But Kommander optimizes this in two ways:

1. **Coalescing.** Multiple concurrent callers share a single in-flight ack round. If 100 clients call `ConfirmLeadershipAsync` at the same time, Kommander sends one round of heartbeats and returns the result to all 100 callers.
2. **Freshness caching.** A confirmation that completed within the last heartbeat interval is reused. Steady-state cost is approximately one quorum round-trip per heartbeat interval, regardless of read volume.

If the confirmation times out (the node cannot reach a quorum within `LeadershipConfirmationTimeout`, default 2 seconds), the method returns `false`. This is the correct behavior for a minority-partitioned leader: it cannot reach a quorum, so the confirmation fails, and your application can redirect the client.

6.2.4 Choosing the Right Check

Method	Cost	Guarantee	Use when
<code>AmILeaderQuick</code>	One volatile read	Local belief	Fast pre-filter; false positive is tolerable
<code>AmILeader</code>	Local state read	Local belief (more current)	Standard write-path gate
<code>ConfirmLeadershipAsync</code>	Quorum round-trip	Quorum-confirmed	Strongly consistent reads (Chapter 6)

For writes, `AmILeader` is sufficient. `ReplicateLogs` has its own safety net: if the node loses leadership between the `AmILeader` check and the replication attempt, the replication fails and returns a non-success status. The client retries.

For reads, the choice depends on your consistency requirements. Chapter 6 covers this in detail.

6.3 Finding the Leader

When a node is not the leader, it needs to tell the client where the leader is. `Kommander` provides two methods for this.

6.3.1 WaitForLeader

```
string leaderEndpoint = await raft.WaitForLeader(  
    partitionId,  
    cancellationToken  
);
```

`WaitForLeader` blocks until a leader is known for the given partition. It returns the leader's endpoint as a string in `host:port` format. If no leader is elected before the cancellation token fires, the method throws.

Use `WaitForLeader` during startup or when you need to wait for the cluster to be ready. It is not suitable for per-request routing because it blocks when there is no leader.

6.3.2 WaitForLeaderStableAsync

```
string leaderEndpoint = await raft.WaitForLeaderStableAsync(  
    partitionId,  
    minStableFor: TimeSpan.FromSeconds(2),  
    timeout: TimeSpan.FromSeconds(10),  
    cancellationToken  
);
```

`WaitForLeaderStableAsync` waits until the same leader has been stable for a minimum duration. This is useful after a leader change: the first elected leader may immediately lose a re-election if another node has a more up-to-date log. Waiting for stability avoids routing to a leader that is about to lose the position.

The `timeout` parameter sets an upper bound on the wait. If leadership keeps churning and never stabilizes, the method throws a `TimeoutException` rather than blocking forever.

6.3.3 GetPartitionLeaderHint

```
string? leaderHint = raft.GetPartitionLeaderHint(partitionId);
```

`GetPartitionLeaderHint` returns the current best-effort leader hint for a partition. It does not block and does not wait. If no leader is known, it returns `null`.

For a locally hosted partition, the hint is the node's own leader belief. For a partition hosted on other nodes, the hint comes from gossiped load reports.

This is the best choice for per-request routing: fast, non-blocking, and good enough for a first attempt. If the hint is wrong, the target node rejects the write, and the client can retry.

6.4 Routing Strategies

6.4.1 Strategy 1: Client-Side Redirect

The simplest approach. The follower returns an HTTP redirect (307 Temporary Redirect with `preserveMethod: true`) to the leader's endpoint. The client follows the redirect automatically.

```
app.MapPut("/store/{key}", async (string key, HttpRequest request) =>
{
    bool isLeader = await raft.AmILeader(partitionId, default);
    if (!isLeader)
    {
        string? leader = raft.GetPartitionLeaderHint(partitionId);
        if (leader is not null)
        {
            string leaderUrl = BuildLeaderUrl(leader, $"/store/{key}");
            return Results.Redirect(leaderUrl, permanent: false, preserveMethod:
true);
        }

        return Results.Json(
            new { error = "No leader available, retry later" },
            statusCode: 503
        );
    }

    // ... handle the write on the leader
});
```

Advantages:

- Simple to implement.

- The client learns the leader's address and can send future requests directly.
- No extra network hop on the server side.

Disadvantages:

- The client must support redirects with method preservation.
- Exposes internal node addresses to the client.

6.4.2 Strategy 2: Server-Side Proxy

The follower proxies the request to the leader using `HttpClient`. The client sees a single endpoint and does not know which node is the leader.

```
app.MapPut("/store/{key}", async (string key, HttpRequest request) =>
{
    bool isLeader = await raft.AmILeader(partitionId, default);
    if (!isLeader)
    {
        string? leader = raft.GetPartitionLeaderHint(partitionId);
        if (leader is not null)
        {
            string leaderUrl = BuildLeaderUrl(leader, $"/store/{key}");

            using var reader = new StreamReader(request.Body);
            string body = await reader.ReadToEndAsync();

            using var client = new HttpClient();
            var response = await client.PutAsync(
                leaderUrl,
                new StringContent(body)
            );

            string content = await response.Content.ReadAsStringAsync();
            return Results.Text(
                content,
                contentType: "application/json",
                statusCode: (int)response.StatusCode
            );
        }

        return Results.Json(
            new { error = "No leader available, retry later" },
            statusCode: 503
        );
    }
}
```

```
// ... handle the write on the leader
});
```

Advantages:

- The client does not need to know about leader routing.
- Internal node addresses stay internal.
- Works behind a load balancer with no special configuration.

Disadvantages:

- Extra network hop: client to follower to leader.
- The follower must parse and re-serialize the request body.
- You must manage `HttpClient` lifetime properly (use `IHttpClientFactory` in production).

6.4.3 Strategy 3: Leader Endpoint in Response Headers

Return the leader's endpoint in a response header. The client uses it for subsequent requests. This avoids redirects while still giving the client enough information to route directly.

```
app.MapPut("/store/{key}", async (string key, HttpContext context) =>
{
    bool isLeader = await raft.AmILeader(partitionId, default);

    string? leader = raft.GetPartitionLeaderHint(partitionId);
    if (leader is not null)
        context.Response.Headers["X-Leader-Endpoint"] = leader;

    if (!isLeader)
    {
        context.Response.StatusCode = 503;
        await context.Response.WriteAsJsonAsync(
            new { error = "Not the leader", leader }
        );
        return;
    }

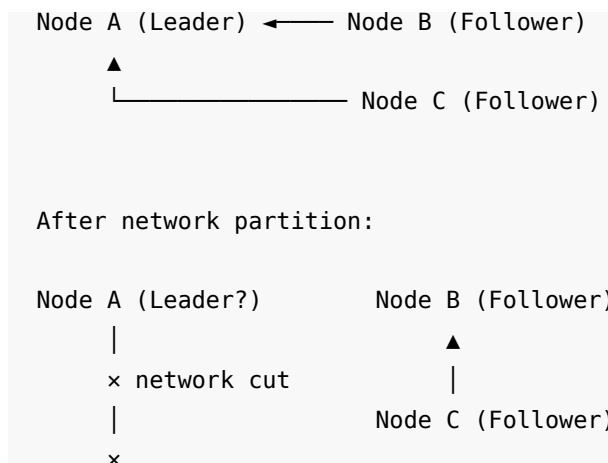
    // ... handle the write on the leader
});
```

The client reads the `X-Leader-Endpoint` header and uses it for the next request. This is a common pattern in distributed databases.

6.5 The Stale Leader Problem

The scenario that makes leadership checks subtle:

Before partition:



Node A was the leader. A network partition isolates Node A from Nodes B and C. On Node A:

- `AmILeader` returns `true`. Node A has not received a higher-term message.
- `AmILeaderQuick` returns `true`. Same reason.
- `ReplicateLogs` accepts the proposal but cannot commit it. There is no quorum (A alone is 1 of 3).
- The client waits for `ReplicateLogs` to return. It eventually times out.

On Nodes B and C:

- Heartbeats from A stop arriving.
- After the election timeout, one of them becomes a candidate and wins.
- The new leader accepts writes. Quorum is 2 of 3 (B and C). Writes commit.

When the partition heals:

- Node A receives a message with a higher term.
- Node A steps down to follower.
- Node A discards any uncommitted entries and catches up from the new leader.

The key insight: writes on Node A during the partition never committed. No data was lost or corrupted. But the client waited for a response that never came. Good routing and timeouts are the application's defense.

6.5.1 How `ConfirmLeadershipAsync` Helps

If Node A calls `ConfirmLeadershipAsync` during the partition, the quorum round fails. The method returns `false` within `LeadershipConfirmationTimeout` (default: 2 seconds). The application knows the leadership is not confirmed and can return a 503 immediately.

This is why `ConfirmLeadershipAsync` matters for reads. A write to a stale leader fails to commit (it times out). But a read from a stale leader returns stale data silently. The read looks correct but returns values that the new leader has already overwritten. `ConfirmLeadershipAsync` prevents this.

6.6 The OnLeaderChanged Event

```
raft.OnLeaderChanged += (partitionId, leaderEndpoint) =>
{
    Console.WriteLine(
        $"Partition {partitionId}: leader is now {leaderEndpoint}"
    );
    return Task.FromResult(true);
};
```

`OnLeaderChanged` fires whenever a new leader is elected for a partition. The callback receives the partition id and the new leader's endpoint.

Use this event for:

- **Logging.** Record leader changes for operational visibility.
- **Cache invalidation.** If your routing logic caches the leader endpoint, invalidate the cache here.
- **Application logic.** Some applications run background tasks only on the leader. Use `OnLeaderChanged` to start or stop those tasks.

The event fires on every node, not just the new leader. Every node in the cluster learns about the leadership change.

6.7 Timing Parameters

Three configuration properties control the timing of leader election:

Property	Default	Purpose
<code>HeartbeatInterval</code>	500 ms	How often the leader sends heartbeats to followers
<code>StartElectionTimeout</code>	2000 ms	Lower bound of the randomized election timeout
<code>EndElectionTimeout</code>	4000 ms	Upper bound of the randomized election timeout

The relationship between these values matters. The Raft paper recommends:

`HeartbeatInterval << ElectionTimeout << MTBF`

Where MTBF is the mean time between failures. The heartbeat must be much shorter than the election timeout so that a temporary network hiccup does not trigger a spurious election. The election timeout must be much shorter than the failure rate so that the cluster recovers before the next failure.

The defaults (500 ms heartbeat, 2000 to 4000 ms election timeout) work for most deployments. If your network has higher latency, increase both values proportionally.

6.8 Putting It Together: The ReplicatedStore Write Handler

Here is the recommended write handler that combines the patterns from this chapter:

```

app.MapPut("/store/{key}", async (string key, HttpRequest request) =>
{
    // Fast local check
    bool isLeader = await raft.AmILeader(partitionId, default);
    if (!isLeader)
    {
        // Try to help the client find the leader
        string? leader = raft.GetPartitionLeaderHint(partitionId);
        return Results.Json(
            new { error = "NotLeader", leader },
            statusCode: 503
        );
    }

    // This node believes it is the leader. Proceed with the write.
    using var reader = new StreamReader(request.Body);
    string value = await reader.ReadToEndAsync();

    byte[] payload = JsonSerializer.SerializeToUtf8Bytes(
        new SetValueCommand(key, value)
    );

    using var timeout = new CancellationTokenSource(TimeSpan.FromSeconds(5));
    var result = await raft.ReplicateLogs(
        partitionId, "SetValue", payload,
        cancellationToken: timeout.Token
    );

    if (result.Success)
        return Results.Ok(new { key, value, logIndex = result.LogIndex });

    // Replication failed. The most common cause is a leadership change.
    if (result.Status == RaftOperationStatus.NodeIsNotLeader)
    {
        string? newLeader = raft.GetPartitionLeaderHint(partitionId);
        return Results.Json(
            new { error = "LeaderChanged", leader = newLeader },
            statusCode: 503
        );
    }

    return Results.Json(
        new { error = result.Status.ToString() },
        statusCode: 503
    );
}

```



```
    );  
};
```

This handler:

1. Checks `AmILeader` first. If not the leader, returns 503 with the leader hint.
2. Attempts the write with a timeout.
3. If `ReplicateLogs` returns `NodeIsNotLeader` (leadership changed between the check and the replication), returns 503 with the new leader hint.
4. For any other failure, returns 503 with the status.

The client reads the `leader` field from the error response and retries against that endpoint.

6.9 Summary

- Only the partition leader can propose writes. Followers must redirect or forward.
- `AmILeaderQuick` is a fast volatile read. It reports local belief. Use it for pre-checks where a false positive is acceptable.
- `AmILeader` reads published local state. It also reports local belief, with slightly more current data. Use it for standard write-path gates.
- `ConfirmLeadershipAsync` contacts a quorum and confirms leadership. Use it for strongly consistent reads (Chapter 6).
- A minority-partitioned leader believes it leads but cannot commit writes. Writes time out. Reads return stale data.
- `GetPartitionLeaderHint` returns a non-blocking best-effort leader hint. Use it for routing.
- `WaitForLeader` blocks until a leader is known. Use it during startup.
- Three routing strategies exist: client-side redirect, server-side proxy, and leader endpoint in headers. Choose based on whether clients need to see internal addresses.
- The `OnLeaderChanged` event fires on every node when a new leader is elected.
- The next chapter uses `ConfirmLeadershipAsync` to implement strongly consistent reads.

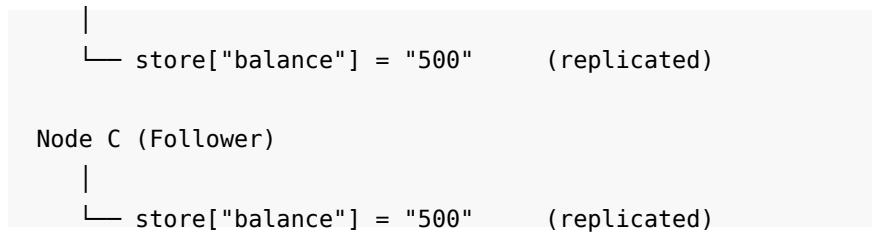
7 Strongly Consistent Reads

In Chapter 5, the read handler read directly from the local dictionary. This works most of the time. But under certain conditions, it returns stale data. This chapter explains why and shows how to fix it.

7.1 The Problem with Local Reads

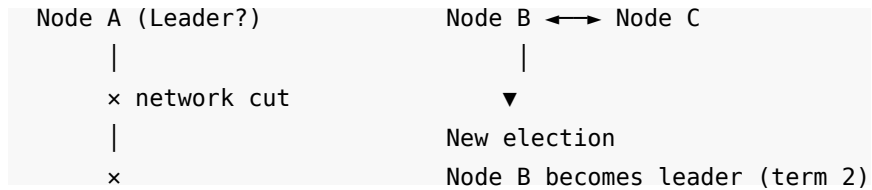
Consider a three-node cluster where Node A is the leader:

```
Node A (Leader)  
  |  
  └─ store["balance"] = "500"      (latest committed value)  
  |  
Node B (Follower)
```

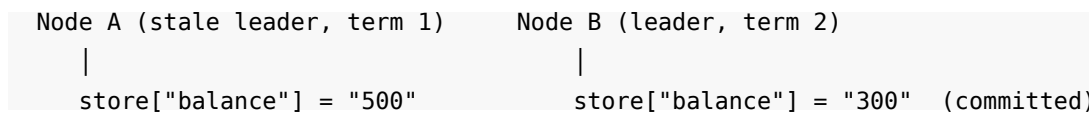


A client reads `GET /store/balance` from Node A and receives "500". This is correct.

Now imagine a network partition isolates Node A from Nodes B and C:



Node B wins a new election. A client writes `balance = "300"` to Node B. The write commits on Nodes B and C (quorum of two).



Now a different client reads `GET /store/balance` from Node A. The read handler checks `AmILeader`. Node A still believes it is the leader (it has not received a higher-term message). The handler reads from the local dictionary and returns "500".

This response is wrong. The committed value is "300". Node A served stale data as if it were the current truth.

7.2 Why Writes Do Not Have This Problem

Writes are safe because `ReplicateLogs` requires a quorum to commit. On the partitioned Node A:

1. The client calls `ReplicateLogs` on Node A.
2. Node A appends the entry to its local WAL.
3. Node A tries to send the entry to Nodes B and C.
4. The network is cut. No follower acknowledges.
5. Quorum is not reached. The write never commits.
6. `ReplicateLogs` times out and returns a failure.

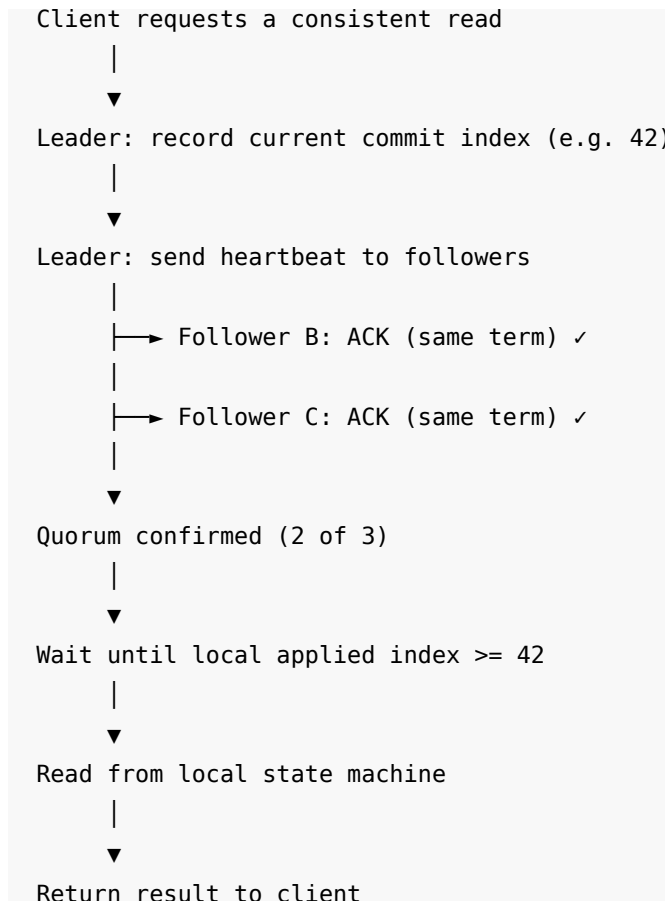
The write path has a built-in safety check: the quorum requirement. The read path, as implemented so far, has no such check. It reads directly from local memory.

7.3 The Read-Index Protocol

The Raft dissertation (section 6.4) describes a protocol for linearizable reads. The idea is simple: before reading local state, confirm that this node is still the leader by contacting a quorum.

The protocol has two steps:

1. **Quorum confirmation.** The leader sends a heartbeat to all followers and waits for a majority to acknowledge. If a majority responds, the leader is still the leader. If not, the leader may have been deposited.
2. **Applied frontier check.** The leader records the commit index at the time of confirmation. It waits until its local state machine has applied all entries up to that index. After that point, a local read reflects all committed writes.



If the leader is partitioned, step 1 fails. The heartbeats do not reach a quorum. The confirmation times out, and the application knows it must not serve the read.

7.4 ConfirmLeadershipAsync

Kommander implements the read-index protocol through `ConfirmLeadershipAsync`:

```
bool confirmed = await raft.ConfirmLeadershipAsync(
    partitionId,
    cancellationToken
);
```

If `confirmed` is true:

- A majority of nodes have acknowledged a heartbeat in the current term.

- All committed entries up to the confirmation point have been applied to the local state machine.
- A read from local state after this call is linearizable.

If `confirmed` is `false`:

- The node could not reach a quorum, or it is not the leader.
- The application must not read from local state. Redirect the client to another node or return an error.

7.4.1 Timeout

The confirmation waits up to `LeadershipConfirmationTimeout` (default: 2 seconds) for the quorum ack round. If the timeout expires, the method returns `false`. This bounds the worst case: a partitioned leader fails the confirmation within 2 seconds rather than blocking forever.

You can configure this timeout:

```
RaftConfiguration configuration = new()
{
    // ...
    LeadershipConfirmationTimeout = TimeSpan.FromSeconds(3)
};
```

Keep this value above one heartbeat round-trip under load.

7.5 Adding Consistent Reads to ReplicatedStore

Add a query parameter to the read endpoint. When the client requests `?consistent=true`, the handler confirms leadership before reading.

```
app.MapGet("/store/{key}", async (string key, HttpRequest request) =>
{
    bool consistent = request.Query.ContainsKey("consistent");

    if (consistent)
    {
        using var timeout = new CancellationTokenSource(
            TimeSpan.FromSeconds(3)
        );

        bool confirmed = await raft.ConfirmLeadershipAsync(
            partitionId,
            timeout.Token
        );

        if (!confirmed)
        {
            string? leader = raft.GetPartitionLeaderHint(partitionId);
```

```

        return Results.Json(
            new { error = "LeadershipNotConfirmed", leader },
            statusCode: 503
        );
    }

    return store.TryGetValue(key, out string? value)
        ? Results.Ok(new { key, value })
        : Results.NotFound();
});

```

The client chooses the consistency level per request:

```

# Fast read (eventual consistency, local state)
curl http://localhost:5001/store/balance

# Consistent read (linearizable, quorum-confirmed)
curl http://localhost:5001/store/balance?consistent=true

```

This pattern gives the application flexibility. Most reads can use the fast path. Only reads that require the latest committed value use the confirmed path.

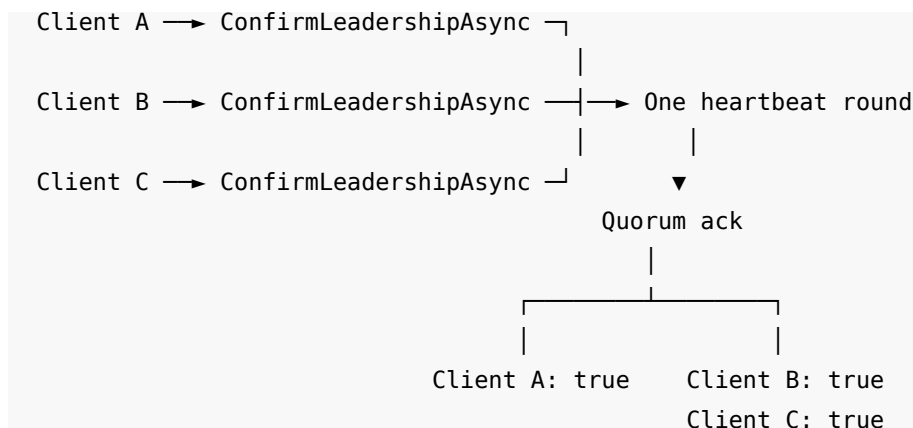
7.6 The Cost of Consistency

A consistent read adds one quorum round-trip compared to a local read. On a three-node cluster in the same data center, this typically adds 0.5 to 2 milliseconds of latency.

Kommander reduces this cost with two optimizations:

7.6.1 Coalescing

If multiple clients request consistent reads at the same time, Kommander sends a single heartbeat round and shares the result with all callers. If 100 clients call `ConfirmLeadershipAsync` within the same heartbeat interval, only one quorum round-trip occurs. All 100 callers receive the same `true` or `false` result.



7.6.2 Freshness Caching

If a quorum confirmation completed within the last heartbeat interval (default: 500 ms), Kommander reuses it. The reasoning: within one heartbeat interval, no other node can assemble an election quorum and take over leadership. A confirmation from 200 ms ago is still valid.

This means that under steady read traffic, the cost is approximately one quorum round-trip per heartbeat interval, regardless of how many reads arrive.

```
T = 0 ms      Confirmation round completes (quorum confirmed)
T = 100 ms    Read arrives → reuses cached confirmation (no round-trip)
T = 200 ms    Read arrives → reuses cached confirmation (no round-trip)
T = 400 ms    Read arrives → reuses cached confirmation (no round-trip)
T = 500 ms    Heartbeat interval expires → next confirmation starts a new round
```

7.7 Consistent Reads on Followers: `ConfirmLocalApplicationAsync`

`ConfirmLeadershipAsync` works only on the leader. If the client sends a consistent read to a follower, the follower cannot confirm its own leadership (it is not the leader).

For cases where you want to read from a follower and still guarantee that the read reflects all committed writes, Kommander provides `ConfirmLocalApplicationAsync`:

```
bool confirmed = await raft.ConfirmLocalApplicationAsync(
    partitionId,
    cancellationToken
);
```

This method works on any node, whether leader or follower. It contacts the leader, obtains the current commit index through a quorum ack round, and waits until the local node has applied all entries up to that index. If it returns `true`, the local state machine includes every committed write.

On the leader, `ConfirmLocalApplicationAsync` is equivalent to `ConfirmLeadershipAsync`. On a follower, it adds a network round-trip to the leader (which then runs its own quorum ack round) plus the time for the follower to apply any entries it has not yet processed.

7.7.1 When to Use Each Method

Method	Node	Guarantee	Use when
<code>ConfirmLeadershipAsync</code>	Leader only	Linearizable	You route reads to the leader
<code>ConfirmLocalApplicationAsync</code>	Any node	Linearizable	You want to read from a follower without forwarding

`ConfirmLocalApplicationAsync` is useful when you want to spread read load across the cluster. Each node can serve consistent reads from its own local state after confirming that it has caught up to the leader's commit index.

7.7.2 Follower Read Example

```
app.MapGet("/store/{key}", async (string key, HttpRequest request) =>
{
    bool consistent = request.Query.ContainsKey("consistent");

    if (consistent)
    {
        using var timeout = new CancellationTokenSource(
            TimeSpan.FromSeconds(3)
        );

        // Works on both leader and follower
        bool confirmed = await raft.ConfirmLocalApplicationAsync(
            partitionId,
            timeout.Token
        );

        if (!confirmed)
        {
            return Results.Json(
                new { error = "ConfirmationFailed" },
                statusCode: 503
            );
        }
    }

    return store.TryGetValue(key, out string? value)
        ? Results.Ok(new { key, value })
        : Results.NotFound();
});
```

With this handler, a load balancer can distribute consistent reads across all three nodes. Each node confirms its own state before serving the read.

7.8 When Not to Use Consistent Reads

Consistent reads are not always necessary. Consider the consistency requirements of each read:

Scenario	Consistency needed	Recommendation
Dashboard showing latest metrics	Eventual	Local read (no confirmation)
User reads their own recent write	Linearizable	<code>ConfirmLeadershipAsync</code> or read-after-write at the application level
Bank balance check before a transfer	Linearizable	<code>ConfirmLeadershipAsync</code>
Search index lookup	Eventual	Local read from any node
Configuration value read by background job	Depends on use case	Local read is usually acceptable

The performance difference is real. A local read is a dictionary lookup: sub-microsecond. A confirmed read adds a quorum round-trip: typically 0.5 to 2 ms in the same data center, longer across regions. For applications that serve thousands of reads per second, this difference matters.

The design pattern: default to local reads, add confirmation where correctness requires it.

7.9 The Complete Stale-Read Scenario

Here is the full timeline of the stale-read problem and how `ConfirmLeadershipAsync` solves it.

7.9.1 Without Confirmation

```

T=0  Node A is leader. balance = "500".
T=1  Network partition isolates A from B, C.
T=2  Node B wins election (term 2).
T=3  Client writes balance = "300" to B. Commits on B, C.
T=4  Client reads from A:
      AmILeader → true (A has not seen term 2)
      Read from local state → "500" (STALE)
T=5  Partition heals. A discovers term 2. Steps down.
      A catches up. balance = "300" on all nodes.

```

Between T=1 and T=5, reads from Node A return stale data. The window can last for the entire duration of the network partition.

7.9.2 With Confirmation

```

T=0  Node A is leader. balance = "500".
T=1  Network partition isolates A from B, C.
T=2  Node B wins election (term 2).
T=3  Client writes balance = "300" to B. Commits on B, C.
T=4  Client reads from A:

```



```
ConfirmLeadershipAsync → sends heartbeat to B, C
B, C unreachable (partition)
Timeout (2 seconds) → returns false
Application returns 503 with leader hint
T=4.1 Client retries on B:
ConfirmLeadershipAsync → quorum ack from C ✓
Read from local state → "300" (CORRECT)
```

The confirmation fails on the partitioned node within 2 seconds. The client receives a clear error and can retry on the correct node.

7.10 Internals: How It Works

The read-index confirmation runs inside the `ReadIndexCoordinator`, one instance per partition. The coordinator manages three concerns:

1. **Round lifecycle.** When a `ConfirmLeadershipAsync` call arrives, the coordinator either joins an existing in-flight round or starts a new one. It triggers a heartbeat to all followers and records the current commit index. When a majority of followers acknowledge (same term), the round succeeds.
2. **Applied frontier wait.** After the quorum confirms, the coordinator checks whether the local state machine has applied all entries up to the recorded commit index. If entries are still being applied, the caller waits. Once the applied index reaches the recorded commit index, the read is safe.
3. **Expiry.** If the quorum ack round does not complete within `LeadershipConfirmationTimeout`, all waiters receive `false`. This prevents unbounded blocking on a partitioned leader.

Late arrivals (callers that arrive while a round is in flight but after the commit index has advanced past the round's recorded index) join a pending queue. They form the next round, which starts as soon as the current one finishes. The timeout budget spans the full wait for each caller, not just its own round.

7.11 Summary

- A local read from the leader is not automatically linearizable. A deposed leader can serve stale data.
- `ConfirmLeadershipAsync` implements the read-index protocol. It proves current leadership by contacting a quorum before the read.
- If confirmation succeeds, a local read after the call is linearizable. If it fails, the application must redirect or retry.
- `ConfirmLocalApplicationAsync` extends the guarantee to follower nodes. It confirms that the follower has applied all committed entries.
- Concurrent callers coalesce into a single quorum round-trip. A recent confirmation is reused within the heartbeat interval.
- The cost is approximately one quorum round-trip per heartbeat interval under steady traffic.

- `LeadershipConfirmationTimeout` (default: 2 seconds) bounds the worst case for a partitioned leader.
- Not every read needs consistency. Default to local reads and add confirmation where correctness requires it.
- The next chapter examines the full lifecycle of a replicated command, from proposal to commit.

8 Replication Deep Dive

The previous chapters used `ReplicateLogs`, checked the `Success` property, and moved on. This chapter opens the hood. It traces a command from the moment your application calls `ReplicateLogs` to the moment `OnReplicationReceived` fires on every node. It explains what happens at each step, what can go wrong, and what the result tells you.

8.1 The Full Lifecycle

When you call `ReplicateLogs`, Kommander executes the following sequence:

1. Admission
 - | Is this node the leader?
 - | Is the proposal queue full?
- ▼
2. Propose
 - | Assign a log position (Id) and a ticket (HLC timestamp)
 - | Write the entry to the local WAL as Proposed
 - | Fsync the WAL
- ▼
3. Replicate
 - | Send `AppendEntries` RPC to each follower
 - | Each follower appends to its own WAL and fsyncs
 - | Each follower sends an acknowledgment back
- ▼
4. Quorum
 - | Count acknowledgments (including the leader itself)
 - | A majority of voters must acknowledge
- ▼
5. Commit
 - | Write the Committed record to the leader's WAL
 - | Send commit notification to followers
- ▼
6. Apply
 - | Fire `OnReplicationReceived` on each node
 - | The application updates its state machine
- ▼
7. Return
 - | Return `RaftReplicationResult` to the caller

Each step has specific guarantees and failure modes. The sections below cover them in order.

8.2 Step 1: Admission

Before Kommander proposes the entry, it checks two conditions:

Is this node the leader? If not, `ReplicateLogs` returns immediately with `Status = NodeIsNotLeader`. No WAL write occurs. No network traffic is sent.

Is the proposal queue full? Each partition has a bounded queue of pending proposals (default: 2048 entries, controlled by `MaxQueuedClientProposalsPerPartition`). If the queue is full, `ReplicateLogs` returns `Status = ProposalQueueFull`. This is backpressure: the partition is accepting proposals faster than it can replicate and commit them. The application should wait and retry.

```
var result = await raft.ReplicateLogs(partitionId, "SetValue", payload);

if (!result.Success)
{
    switch (result.Status)
    {
        case RaftOperationStatus.NodeIsNotLeader:
            // Redirect to the leader
            break;

        case RaftOperationStatus.ProposalQueueFull:
            // Back off and retry after a delay
            break;
    }
}
```

8.3 Step 2: Propose

The leader assigns the entry a log position (the `Id` field on `RaftLog`) and a proposal ticket (an `HLCTimestamp`). The log position is monotonically increasing within the partition. The ticket identifies this specific proposal through the rest of the lifecycle.

The leader then writes the entry to its local WAL with type `Proposed` and fsyncs the write. The fsync is the durability point on the leader. After this point, the entry survives a leader crash: when the leader restarts, it reads the proposed entry from the WAL and can re-replicate it.

8.3.1 The Propose is the Durability Point

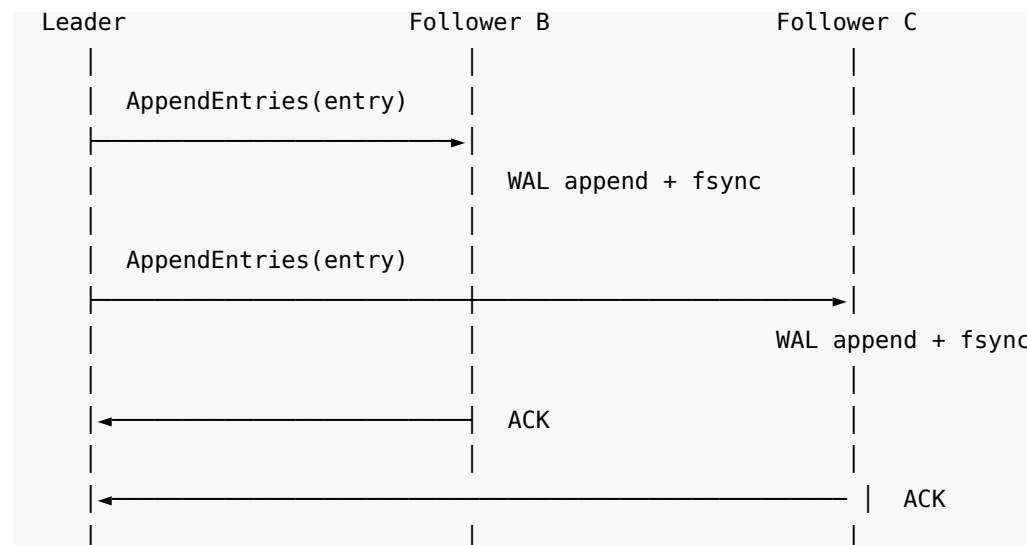
This is important enough to emphasize. The WAL fsync after the propose is the moment the entry becomes durable on the leader. If the leader crashes before the fsync, the entry is lost (but the client has not received a success response, so no data is silently lost). If the leader crashes after the fsync, the entry is on disk and will be recovered.

8.4 Step 3: Replicate

The leader sends the entry to each follower using the `AppendEntries` RPC. This is the Raft protocol's replication mechanism. The message includes:

- The entry itself (log position, term, type, data).
- The leader's term.
- The previous log position and term (for the Log Matching Property, which ensures followers have the same log prefix as the leader).

Each follower receives the entry, validates it, appends it to its own WAL, and fsyncs. The follower then sends an acknowledgment back to the leader.



The leader does not wait for all followers to acknowledge. It waits for a quorum.

8.5 Step 4: Quorum

A quorum is a majority of the cluster's voters. For a three-node cluster, the quorum is 2. The leader counts itself as one of the acknowledgers (it already wrote the entry to its own WAL). So the leader needs only one follower acknowledgment to reach quorum.

Cluster size	Quorum	Follower ACKs needed
3 nodes	2	1
5 nodes	3	2
7 nodes	4	3

As soon as the quorum is reached, the entry is considered committed. This can happen before all followers have acknowledged. In a three-node cluster, the entry commits as soon as the first follower responds, even if the second follower has not responded yet.

The second follower will receive the entry eventually (through a later `AppendEntries` or a catch-up mechanism). But the commit decision does not wait for it.

8.6 Step 5: Commit

After quorum is reached, the leader writes a `Committed` record to its own WAL. It also notifies the followers that the entry is committed (via the `leaderCommitIndex` field in subsequent `AppendEntries` messages).

8.6.1 The Single-Fsync Fast Path

By default, Kommander uses the **single-fsync commit** path (`WalSingleFsyncCommit = true`). In this mode, the client's `ReplicateLogs` call returns as soon as the propose quorum is durable (the leader and at least one follower have fsynced the Proposed entry). The `Committed` record is written to the WAL afterward, but the client does not wait for that second fsync.

This is safe because the propose-quorum durability is the true Raft commit point. A quorum of nodes holds the entry on disk. The `Committed` record is bookkeeping: it records a fact that is already guaranteed by the quorum.

The benefit is latency: the client's critical path contains one fsync instead of two. For workloads where WAL fsync dominates the latency (typically 1 to 10 ms per fsync on SSD), this removes a significant chunk.

You can disable this fast path:

```
RaftConfiguration configuration = new()  
{  
    // ...  
    WalSingleFsyncCommit = false  
};
```

With `WalSingleFsyncCommit = false`, the client waits for both the Proposed fsync and the `Committed` fsync. This means the on-disk WAL is always a reliable record of what is committed. The trade-off is higher write latency.

8.7 Step 6: Apply

After the entry is committed, Kommander fires `OnReplicationReceived` on each node that holds the partition. The callback receives the partition id and the `RaftLog` object.

The callback fires in log order. Entry 41 is applied before entry 42. If entry 42 arrives before entry 41 (due to reordering in the replication path), Kommander holds entry 42 until entry 41 is applied.

On the leader, the callback fires after the commit is confirmed. On followers, the callback fires after the follower learns that the entry is committed (via the leader's commit index in a subsequent heartbeat or `AppendEntries`).

8.8 Step 7: Return

`ReplicateLogs` returns a `RaftReplicationResult` with four properties:

```
RaftReplicationResult result = await raft.ReplicateLogs(  
    partitionId, "SetValue", payload
```

```
);

// result.Success    true if the entry was committed
// result.Status     the specific outcome
// result.LogIndex   the log position assigned to this entry
// result.TicketId   the HLC timestamp that identifies this proposal
```

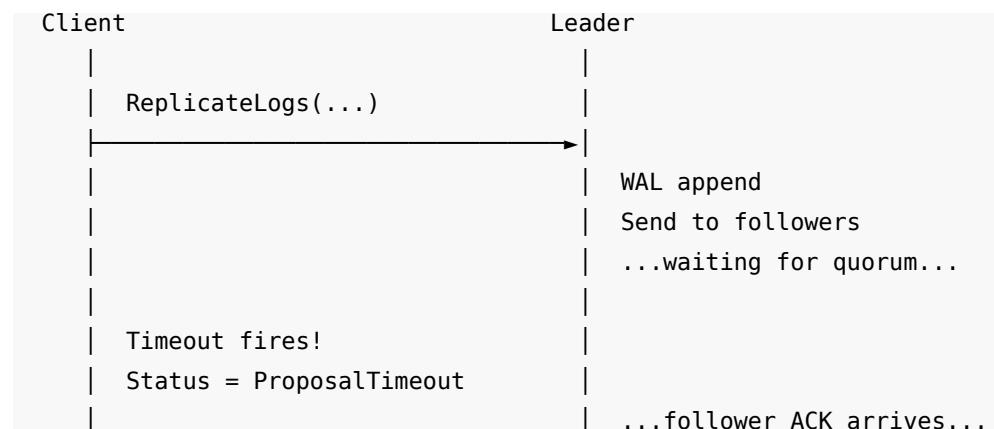
8.8.1 RaftOperationStatus Values

The `Status` field tells you exactly what happened. The most common values:

Status	Meaning	Action
Success	The entry was committed.	None.
NodeIsNotLeader	This node is not the leader.	Redirect to the leader.
ProposalQueueFull	The partition's proposal queue is at capacity.	Back off and retry.
ProposalTimeout	The quorum ack did not arrive in time.	The entry may or may not have committed (see below).
OperationCancelled	The caller's cancellation token fired.	The entry may or may not have committed (see below).
RestoreInProgress	The partition is still re-playing WAL entries.	Wait and retry.
PartitionMoved	The partition generation changed (split/merge).	Refresh the partition map and retry.

8.9 The Ambiguity Window

Two statuses require special attention: `ProposalTimeout` and `OperationCancelled`. Both indicate that the client gave up waiting, but they do not tell you whether the entry was committed.



Client thinks: "failed"	Quorum reached → committed
	OnReplicationReceived fires

The client received a timeout. The client does not know the outcome. The entry might have been committed a moment after the timeout fired. Or the entry might never have reached quorum.

This is the **ambiguity window**. It exists in every distributed system that separates the client from the commit decision. The network can fail between the commit and the client's receipt of the result.

8.9.1 Handling the Ambiguity

There are two strategies:

1. Idempotent commands. Design commands so that applying them twice produces the same result as applying them once. `SetValue("balance", "500")` is idempotent: applying it twice still sets the balance to 500. `IncrementCounter("visits")` is not idempotent: applying it twice adds 2 instead of 1. Chapter 8 covers this in detail.

2. Check before retry. After a timeout, read the current state to determine whether the command was applied. If `balance` is already "500", the command committed. If not, retry.

The recommended approach is idempotent commands. They make retries safe without extra reads.

8.10 ReplicateLogs Overloads

Kommander provides several overloads of `ReplicateLogs`:

8.10.1 Single entry

```
Task<RaftReplicationResult> ReplicateLogs(
    int partitionId,
    string type,
    byte[] data,
    bool autoCommit = true,
    long expectedGeneration = 0,
    CancellationToken cancellationToken = default
);
```

This is the most common form. It proposes a single entry with a type label and a byte array payload.

8.10.2 Multiple entries (same type)

```
Task<RaftReplicationResult> ReplicateLogs(
    int partitionId,
    string type,
    IEnumerable<byte[]> logs,
    bool autoCommit = true,
    long expectedGeneration = 0,
```

```

    CancellationToken cancellationToken = default
);

```

This proposes multiple entries in a single batch. All entries share the same type label. They are replicated and committed together. The `OnReplicationReceived` callback fires once for each entry, in order.

Batch replication reduces overhead. Instead of one `AppendEntries` round-trip per entry, the batch sends all entries in one message. For workloads that produce many small entries, batching improves throughput. Chapter 10 covers batch replication in detail.

8.10.3 The `autoCommit` Parameter

The `autoCommit` parameter defaults to `true`. When true, Kommander commits the entry automatically after the quorum confirms it. This is the standard behavior for most applications.

When `autoCommit` is `false`, Kommander proposes and replicates the entry but does not commit it. The entry sits in a `Proposed` state on all nodes. Your application must call `CommitLogs` or `RollbackLogs` to finalize it.

```

// Propose without committing
var result = await raft.ReplicateLogs(
    partitionId, "Transfer", payload,
    autoCommit: false
);

if (result.Success)
{
    // Decide whether to commit or roll back
    bool shouldCommit = ValidateTransfer(result.TicketId);

    if (shouldCommit)
    {
        await raft.CommitLogs(partitionId, result.TicketId);
    }
    else
    {
        await raft.RollbackLogs(partitionId, result.TicketId);
    }
}

```

Manual commit is useful for two-phase operations where the application needs to validate or coordinate before finalizing. Chapter 10 covers this in full.

8.10.4 The `expectedGeneration` Parameter

When the cluster uses elastic partitions (Part III), partitions can split or merge. Each split or merge increments the partition's generation number. The `expectedGeneration` parameter lets you detect when a partition has moved:


```

var result = await raft.ReplicateLogs(
    partitionId, "SetValue", payload,
    expectedGeneration: 3
);

if (result.Status == RaftOperationStatus.PartitionMoved)
{
    // The partition has split or merged since generation 3.
    // Refresh the partition map and retry.
}

```

When `expectedGeneration` is 0 (the default), the check is disabled. Chapter 15 covers generation fencing.

8.11 Observing the Lifecycle

Add logging to the `ReplicatedStore` to trace each step:

```

raft.OnReplicationReceived += (partitionId, log) =>
{
    Console.WriteLine(
        $"[APPLY] partition={partitionId} " +
        $"id={log.Id} type={log.LogType} " +
        $"term={log.Term}"
    );

    if (log.LogType == "SetValue")
    {
        var command = JsonSerializer.Deserialize<SetValueCommand>(
            log.LogData ?? []
        );

        if (command is not null)
            store[command.Key] = command.Value;
    }

    return Task.FromResult(true);
};

raft.OnReplicationError += (partitionId, log) =>
{
    Console.WriteLine(
        $"[ERROR] partition={partitionId} " +
        $"id={log.Id} type={log.LogType}"
    );
};

```

`OnReplicationError` fires when a committed entry fails to apply (the `OnReplicationReceived` callback returned `false` or threw an exception). This is a diagnostic signal. In a healthy system, it should never fire. If it does, investigate why the callback failed.

8.11.1 Reading the Commit Index

You can query the current commit index for a partition:

```
long commitIndex = raft.GetCommitIndex(partitionId);
Console.WriteLine($"Partition {partitionId} commit index: {commitIndex}");
```

`GetCommitIndex` returns the highest log id that is known to be committed on this node. It is a local in-memory read. The commit index increases monotonically as entries are committed.

This is useful for diagnostics and monitoring. A follower whose commit index lags behind the leader's commit index is catching up. A commit index that stops advancing indicates a problem (no leader, no quorum, or a stuck partition).

8.12 Timeline: Three Nodes Processing One Write

Here is a concrete timeline of a write on a three-node cluster:

Time	Leader (A)	Follower (B)	Follower (C)
T=0	ReplicateLogs called		
T=1	WAL append (Proposed) fsync		
T=2	Send AppendEntries →	Receive entry	
		→	Receive entry
T=3		WAL append + fsync Send ACK →	
T=4	Receive ACK from B Quorum: A + B = 2/3 ✓		
T=5	Commit (WAL write) OnReplicationReceived Return Success to client		
T=6			WAL append + fsync Send ACK →
T=7	Receive ACK from C		(late, but not needed)
T=8		Learns commit via next heartbeat	Learns commit via next heartbeat
T=9		OnReplicationReceived	OnReplicationReceived

Key observations:

- The client receives a response at T=5, after quorum (T=4) and the leader's commit.
- Follower C's ACK arrives at T=7, after the entry is already committed. This is fine. The quorum only needed A and B.

- Followers B and C learn about the commit through the next heartbeat (T=8) and apply the entry (T=9).
- The leader's `OnReplicationReceived` fires at T=5. The followers' callbacks fire at T=9. The delay is the heartbeat interval (default: 500 ms).

8.13 What Happens When Things Go Wrong

8.13.1 Leader crashes after propose, before quorum

The entry is in the leader's WAL (fsynced at T=1). No follower received it. If A restarts as leader, it re-replicates the entry. If B or C becomes the new leader, the entry is not in their logs. The new leader's log takes precedence: A's proposed entry is rolled back when A catches up from the new leader. The client received no response (A crashed), so the client retries.

8.13.2 Leader crashes after quorum, before the client receives the response

The entry is committed (a quorum holds it). The client never received the response. The client sees a timeout. The entry is applied on all nodes. The client is in the ambiguity window.

8.13.3 Follower crashes before acknowledging

The leader waits for a quorum. If the other follower acknowledges, quorum is reached (2 of 3). The crashed follower catches up when it restarts. If both followers crash, quorum cannot be reached. The write times out.

8.13.4 Network partition between leader and one follower

The leader still has a quorum (itself + the reachable follower). Writes continue. The unreachable follower catches up when the partition heals.

8.14 Summary

- `ReplicateLogs` proposes a command that goes through admission, propose, replicate, quorum, commit, and apply.
- The propose WAL fsync is the durability point on the leader. After this point, the entry survives a leader crash.
- Quorum means a majority of voters acknowledged. The leader counts itself.
- The single-fsync fast path (`WalSingleFsyncCommit = true`, the default) releases the client after propose-quorum durability, skipping the commit fsync. This reduces write latency without weakening durability.
- `RaftReplicationResult.Status` tells you the exact outcome. Always check it.
- `ProposalTimeout` and `OperationCancelled` are ambiguous: the entry may or may not have committed. Design idempotent commands to handle safe retries.
- `OnReplicationReceived` fires on all nodes, in log order, after the entry is committed. The leader fires it first; followers fire it after the next heartbeat.
- `OnReplicationError` fires when the callback fails. Investigate any occurrence.
- `GetCommitIndex` returns the local commit frontier. Use it for diagnostics.

- The next chapter covers how to design application state machines that process replicated commands correctly.

9 Designing Application State Machines

Kommander replicates commands. Your application interprets them. The callback that processes each committed command is your **state machine**. If the state machine is correct, every node in the cluster arrives at the same state. If the state machine is incorrect, the nodes diverge silently, and no amount of Raft correctness will save you.

This chapter teaches the rules for building a correct state machine. The rules are simple but the consequences of breaking them are severe.

9.1 The Core Rule: Determinism

The state machine must be **deterministic**. Given the same sequence of commands in the same order, every node must produce exactly the same state. No exceptions.

Raft guarantees that every node receives the same commands in the same order. That guarantee is worthless if the nodes interpret those commands differently.

```
Log (same on every node):
```

```
Position 1: SetValue("name", "Alice")
Position 2: IncrementCounter("visits")
Position 3: SetValue("role", "admin")
Position 4: DeleteValue("temp")
```

```
State machine result (must be identical on every node):
```

```
name    = "Alice"
visits  = 1
role    = "admin"
temp    = (deleted)
```

If Node A processes position 2 and gets `visits = 1`, but Node B processes the same command and gets `visits = 2`, the replicas have diverged. Raft cannot detect or correct this. The divergence is permanent.

9.2 What Breaks Determinism

Three patterns break determinism. Each is easy to introduce and hard to detect.

9.2.1 Pattern 1: Reading External State

The state machine reads something that is not part of the command: the current time, a random number, an environment variable, a file on disk, or a value from an external API.

Incorrect:

```
Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
```

```

    if (log.LogType == "SetValue")
    {
        var command = Deserialize<SetValueCommand>(log.LogData);

        // BUG: DateTime.UtcNow is different on each node
        // and different during replay
        store[command.Key] = command.Value;
        store[command.Key + ":updated"] = DateTime.UtcNow.ToString("o");
    }

    return Task.FromResult(true);
}

```

Node A processes this at 14:00:01.123. Node B processes it at 14:00:01.456 (network delay). The ":updated" value differs between nodes.

On replay after restart, Node A processes it at 15:30:00.000. The ":updated" value differs from the live value.

Correct: Include the timestamp in the command itself, set by the client or the API handler before replication.

```

record SetValueCommand(string Key, string Value, string UpdatedAt);

// In the API handler (before replication):
var command = new SetValueCommand(
    key,
    value,
    DateTime.UtcNow.ToString("o") // captured once, replicated to all nodes
);

byte[] payload = JsonSerializer.SerializeToUtf8Bytes(command);
await raft.ReplicateLogs(partitionId, "SetValue", payload);

// In the state machine (same on all nodes, same on replay):
Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
    if (log.LogType == "SetValue")
    {
        var command = Deserialize<SetValueCommand>(log.LogData);
        store[command.Key] = command.Value;
        store[command.Key + ":updated"] = command.UpdatedAt;
    }

    return Task.FromResult(true);
}

```

Now the timestamp is part of the replicated command. Every node applies the same value.

The same rule applies to:

- **Random numbers.** Generate them in the API handler and include them in the command.
- **UUIDs/GUIDs.** Generate them before replication.
- **Configuration values.** If the state machine depends on a configuration setting, replicate the setting as a command so that all nodes use the same value.

9.2.2 Pattern 2: Side Effects in the Callback

The state machine sends an email, calls a webhook, writes to a file, or contacts an external API.

Incorrect:

```
Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
    if (log.LogType == "CreateOrder")
    {
        var order = Deserialize<CreateOrderCommand>(log.LogData);
        orders[order.Id] = order;

        // BUG: fires on every node (3 emails for one order)
        // BUG: fires again on replay after restart
        emailService.SendOrderConfirmation(order.Email, order.Id);
    }

    return Task.FromResult(true);
}
```

This code has two problems:

1. **Multiple executions.** The callback fires on every node. A three-node cluster sends three emails for one order.
2. **Replay.** When a node restarts, `OnLogRestored` replays every committed command. Every historical order triggers another email.

Correct: Separate state changes from side effects. The callback updates state only. Side effects go into a separate queue that runs outside the callback.

```
// Side-effect queue (not replayed)
var pendingNotifications = new ConcurrentQueue<OrderNotification>();
bool isRestoring = false;

raft.OnRestoreStarted += (partitionId) =>
{
    isRestoring = true;
```

```

};

raft.OnRestoreFinished += (partitionId) =>
{
    isRestoring = false;
};

Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
    if (log.LogType == "CreateOrder")
    {
        var order = Deserialize<CreateOrderCommand>(log.LogData);
        orders[order.Id] = order;

        // Queue the side effect only during live application on the leader
        if (!isRestoring)
        {
            pendingNotifications.Enqueue(
                new OrderNotification(order.Email, order.Id)
            );
        }
    }

    return Task.FromResult(true);
}

raft.OnReplicationReceived += ApplyCommand;
raft.OnLogRestored += ApplyCommand;

```

A separate background task processes the notification queue. Only the leader runs this task. If the leader changes, the new leader picks up from where its state machine left off.

9.2.3 Pattern 3: Conditional Logic Based on Node Identity

The state machine behaves differently depending on which node is running it.

Incorrect:

```

Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
    if (log.LogType == "SetValue")
    {
        var command = Deserialize<SetValueCommand>(log.LogData);
        store[command.Key] = command.Value;

        // BUG: only the leader tracks metrics
        // other nodes have different state
        if (await raft.AmILeader(partitionId, default))

```

```

    {
        store["metrics:writes"] =
            (int.Parse(store.GetValueOrDefault("metrics:writes", "0")) + 1)
            .ToString();
    }
}

return Task.FromResult(true);
}

```

The `metrics:writes` counter exists only on the leader. If leadership changes, the new leader starts from zero (or from whatever stale value it had). The nodes have different state.

Correct: If the metrics counter is part of the state, update it on every node. If it is a local-only diagnostic, do not store it in the replicated state.

9.3 The Two Callbacks

Kommander has two callbacks for applying commands. Both must use the same logic.

9.3.1 OnReplicationReceived

```

raft.OnReplicationReceived += (partitionId, log) =>
{
    // Fires for each newly committed command during normal operation.
    return ApplyCommand(partitionId, log);
};

```

This event fires during normal operation. When a command is committed, each node's `OnReplicationReceived` fires with the committed entry.

9.3.2 OnLogRestored

```

raft.OnLogRestored += (partitionId, log) =>
{
    // Fires for each persisted committed command during startup replay.
    return ApplyCommand(partitionId, log);
};

```

This event fires during startup. When a node starts with a durable WAL (RocksDB or SQLite), Kommander reads all committed entries and fires `OnLogRestored` for each one. This is how the application rebuilds its state machine.

9.3.3 The Pattern: One Function, Two Registrations

```

Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
    // Pure state change. No side effects.
    // Same result every time, regardless of context.
    switch (log.LogType)
    {

```



```

        case "SetValue":
            var set = Deserialize<SetValueCommand>(log.LogData);
            store[set.Key] = set.Value;
            break;

        case "DeleteValue":
            var del = Deserialize<DeleteValueCommand>(log.LogData);
            store.TryRemove(del.Key, out _);
            break;

        case "IncrementCounter":
            var inc = Deserialize<IncrementCounterCommand>(log.LogData);
            int current = int.Parse(
                store.GetValueOrDefault(inc.Key, "0")
            );
            store[inc.Key] = (current + 1).ToString();
            break;
    }

    return Task.FromResult(true);
}

raft.OnReplicationReceived += ApplyCommand;
raft.OnLogRestored += ApplyCommand;

```

The function is the same for both events. The behavior is identical. Replay produces the same state as live application.

9.3.4 OnRestoreStarted and OnRestoreFinished

Two additional events bracket the restore phase:

```

raft.OnRestoreStarted += (partitionId) =>
{
    Console.WriteLine($"Partition {partitionId}: restore started");
};

raft.OnRestoreFinished += (partitionId) =>
{
    Console.WriteLine($"Partition {partitionId}: restore complete");
};

```

`OnRestoreStarted` fires before the first `OnLogRestored` call for a partition. `OnRestoreFinished` fires after the last one. Use these to:

- Set a flag that distinguishes replay from live application (as shown in the side-effect example above).
- Initialize or finalize data structures before and after replay.

- Log the restore duration for operational monitoring.

Both events receive the partition id as an `int`. They are `Action<int>` delegates (not `async`).

9.4 Command Serialization

Commands travel as byte arrays through the Raft log. The application must serialize commands before replication and deserialize them in the callback.

9.4.1 Choosing a Serialization Format

Format	Pros	Cons
<code>System.Text.Json</code>	Built into .NET, human-readable, good for debugging	Larger payloads, slower than binary
<code>MessagePack</code>	Compact, fast, schema-based	Requires a NuGet package, not human-readable
<code>Protobuf</code>	Compact, fast, cross-language	Requires .proto files or annotations
Raw bytes	Minimal overhead	Manual parsing, easy to get wrong

For most applications, `System.Text.Json` is the right starting point. The `ReplicatedStore` examples in this book use it. Switch to a binary format when payload size or serialization speed becomes a bottleneck.

9.4.2 The `LogType` Field

Every `ReplicateLogs` call includes a `type` string parameter. This becomes the `LogType` field on the `RaftLog` object. The state machine uses it to dispatch to the correct handler.

```
switch (log.LogType)
{
    case "SetValue":
        // ...
        break;
    case "DeleteValue":
        // ...
        break;
}
```

Keep type strings short and stable. They are persisted in the WAL and replayed on every restart. If you rename a type, old entries in the WAL will not match the new name.

9.5 Command Versioning

Over time, the structure of your commands will change. You might add a field, remove a field, or change a field's type. The WAL contains entries from every version of your application. The state machine must handle all of them.

9.5.1 Strategy 1: Tolerant Deserialization

Use a deserializer that ignores unknown fields and uses defaults for missing fields:

```
var options = new JsonSerializerOptions
{
    PropertyNameCaseInsensitive = true,
    DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull
};

var command = JsonSerializer.Deserialize<SetValueCommand>(
    log.LogData ?? [],
    options
);
```

This handles most additions gracefully. A new field added in version 2 defaults to null when replaying a version 1 entry.

9.5.2 Strategy 2: Explicit Version Field

Include a version number in the command:

```
record SetValueCommand(
    string Key,
    string Value,
    int Version = 1,
    string? UpdatedAt = null // added in version 2
);
```

The state machine checks the version and adjusts its behavior:

```
if (command.Version >= 2 && command.UpdatedAt is not null)
{
    store[command.Key + ":updated"] = command.UpdatedAt;
}
```

9.5.3 Strategy 3: Separate LogType per Version

Use a different LogType for each version: "SetValue.v1", "SetValue.v2". This makes the dispatch explicit but increases the number of cases in the switch statement.

The recommended approach: start with tolerant deserialization. Add explicit versions when the command structure changes in a way that tolerant deserialization cannot handle (such as changing a field's semantics).

9.6 Idempotency

A command is **idempotent** if applying it once produces the same result as applying it two or more times. Idempotent commands are safe to retry after a timeout or a leader change.

9.6.1 Naturally Idempotent Commands

Some commands are idempotent by nature:

- `SetValue("balance", "500")`: applying it twice still sets the balance to 500.
- `DeleteValue("temp")`: deleting a key that does not exist is a no-op.

9.6.2 Non-Idempotent Commands

Other commands are not idempotent:

- `IncrementCounter("visits")`: applying it twice adds 2 instead of 1.
- `AppendToList("log", "entry-42")`: applying it twice adds two copies.
- `Transfer("A", "B", 100)`: applying it twice transfers 200 instead of 100.

9.6.3 Making Commands Idempotent

Add a unique operation id to each command. The state machine tracks which operations have been applied and skips duplicates.

```
record TransferCommand(  
    string OperationId,    // unique per operation  
    string FromAccount,  
    string ToAccount,  
    decimal Amount  
);  
  
var appliedOperations = new HashSet<string>();  
  
Task<bool> ApplyCommand(int partitionId, RaftLog log)  
{  
    if (log.LogType == "Transfer")  
    {  
        var cmd = Deserialize<TransferCommand>(log.LogData);  
  
        // Skip if already applied  
        if (!appliedOperations.Add(cmd.OperationId))  
            return Task.FromResult(true);  
  
        decimal fromBalance = GetBalance(cmd.FromAccount);  
        decimal toBalance = GetBalance(cmd.ToAccount);  
  
        store[cmd.FromAccount] = (fromBalance - cmd.Amount).ToString();  
        store[cmd.ToAccount] = (toBalance + cmd.Amount).ToString();  
    }  
}
```

```

    return Task.FromResult(true);
}

```

The client generates the `OperationId` (typically a GUID) and includes it in every retry of the same operation. The state machine applies the command the first time and skips it on duplicates.

This set of applied operation ids is part of the state machine. It must be rebuilt during replay and must produce the same result on every node.

9.6.4 Managing the Operation Set Size

The set of applied operations grows without bound if you never remove entries. Two strategies:

1. **Time-based expiration.** Remove operations older than a retention period (e.g., 24 hours). Include a timestamp in the command for this purpose. This works if clients do not retry after the retention period.
2. **Sliding window.** Track only the last N operations per client. This bounds memory usage but requires each client to have a stable identity.

9.7 Building the ReplicatedStore State Machine

Here is the evolved state machine that handles multiple command types correctly:

```

// Command records
record SetValueCommand(string Key, string Value, string? UpdatedAt = null);
record DeleteValueCommand(string Key);
record IncrementCounterCommand(string Key);

// State
var store = new ConcurrentDictionary<string, string>();
bool isRestoring = false;

raft.OnRestoreStarted += (_) => isRestoring = true;
raft.OnRestoreFinished += (_) => isRestoring = false;

Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
    switch (log.LogType)
    {
        case "SetValue":
        {
            var cmd = JsonSerializer.Deserialize<SetValueCommand>(
                log.LogData ?? []
            );
            if (cmd is not null)
            {
                store[cmd.Key] = cmd.Value;
            }
        }
    }
}

```

```

        if (cmd.UpdatedAt is not null)
            store[cmd.Key + ":updated"] = cmd.UpdatedAt;
    }
    break;
}

case "DeleteValue":
{
    var cmd = JsonSerializer.Deserialize<DeleteValueCommand>(
        log.LogData ?? []
    );
    if (cmd is not null)
    {
        store.TryRemove(cmd.Key, out _);
        store.TryRemove(cmd.Key + ":updated", out _);
    }
    break;
}

case "IncrementCounter":
{
    var cmd = JsonSerializer.Deserialize<IncrementCounterCommand>(
        log.LogData ?? []
    );
    if (cmd is not null)
    {
        int current = int.Parse(
            store.GetValueOrDefault(cmd.Key, "0")
        );
        store[cmd.Key] = (current + 1).ToString();
    }
    break;
}
}

return Task.FromResult(true);
}

raft.OnReplicationReceived += ApplyCommand;
raft.OnLogRestored += ApplyCommand;

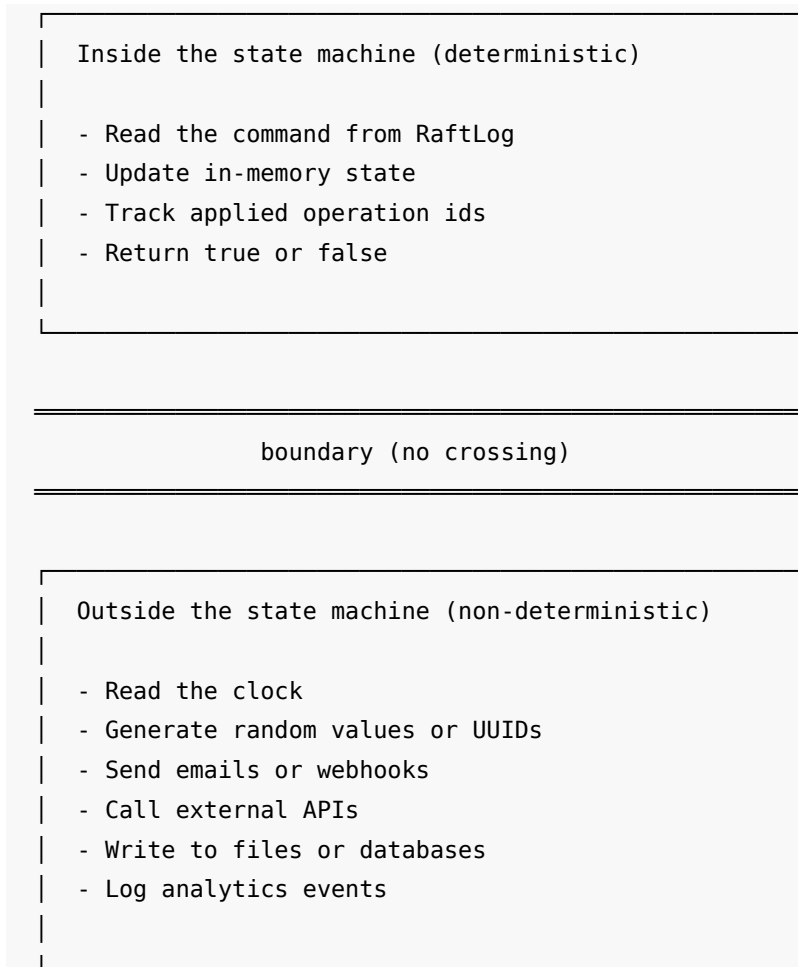
```

This state machine:

- Handles three command types.
- Uses one function for both live application and replay.
- Does not read external state (no `DateTime.UtcNow`, no random values).

- Does not trigger side effects.
- Ignores unknown command types (the `default` case falls through silently).
- Uses tolerant deserialization (missing `UpdatedAt` defaults to `null`).

9.8 The Boundary Diagram



Everything above the boundary is the same on every node and on every replay. Everything below the boundary happens once, on one node, outside the callback.

9.9 Checklist

Before shipping a state machine, verify each of these:

1. Does the callback read `DateTime.UtcNow`, `Random`, or `Guid.NewGuid()`? If yes, move the value into the command payload.
2. Does the callback send an email, call an API, or write to a file? If yes, move it to a post-commit queue outside the callback.
3. Does the callback check `AmILeader` or node identity? If yes, remove the check. The state machine must be the same on every node.
4. Does the callback depend on data that is not in the command or in the state machine? If yes, replicate that data as its own command.

5. Is the command idempotent? If not, add an operation id and track applied operations.
6. Can old command formats still be deserialized? If not, add version handling.

9.10 Summary

- The state machine must be deterministic. Same commands in the same order must produce the same state on every node.
- Never read external state (time, random, environment) inside the apply callback. Include those values in the command payload.
- Never trigger side effects (IO, HTTP, email) inside the callback. Use a separate queue processed outside the callback.
- `OnReplicationReceived` and `OnLogRestored` must use the same function. Replay must produce the same state as live application.
- `OnRestoreStarted` and `OnRestoreFinished` bracket the replay phase. Use them to distinguish replay from live operation when controlling side effects.
- Design commands to be idempotent. Add operation ids for commands that are not naturally idempotent.
- Version your command format. Use tolerant deserialization and explicit version fields as the format evolves.
- The next chapter covers failure modes and retry strategies for distributed operations.

10 Batch Replication and Manual Commit Control

Every `ReplicateLogs` call in the previous chapters sent one command. One command means one proposal, one quorum round-trip, and one WAL flush. For a single write, this is fine. For a bulk import of 10,000 keys, it means 10,000 round-trips.

This chapter shows two ways to reduce that cost:

1. **Batch replication.** Send multiple entries in one proposal. One round-trip. One flush.
2. **Manual commit.** Propose entries without committing them. Validate, then commit or roll back as a unit.

10.1 Batch Replication: Homogeneous

The simplest form of batching sends multiple payloads of the same type in one call. Instead of passing a single `byte[]`, pass a list of `byte[]`.

10.1.1 Single-Entry Call (Before)

```
foreach (var item in items)
{
    byte[] payload = JsonSerializer.SerializeToUtf8Bytes(
        new SetValueCommand(item.Key, item.Value)
    );
}
```



```
await raft.ReplicateLogs(partitionId, "SetValue", payload);
}
```

This loop sends N proposals. Each proposal waits for a quorum before the next one starts. For 100 items on a 1 ms round-trip, the total time is at least 100 ms.

10.1.2 Batch Call (After)

```
List<byte[]> payloads = items.Select(item =>
    JsonSerializer.SerializeToUtf8Bytes(
        new SetValueCommand(item.Key, item.Value)
    )
).ToList();

RaftReplicationResult result = await raft.ReplicateLogs(
    partitionId, "SetValue", payloads
);
```

This call sends all entries in one proposal. One quorum round-trip. One WAL flush. For 100 items on a 1 ms round-trip, the total time is approximately 1 ms.

10.1.3 The Batch Overload

```
Task<RaftReplicationResult> ReplicateLogs(
    int partitionId,
    string type,
    IReadOnlyList<byte[]> logs,
    bool autoCommit = true,
    long expectedGeneration = 0,
    CancellationToken cancellationToken = default
);
```

The parameters are the same as the single-entry overload, except that `logs` is a list instead of a single byte array. All entries in the batch share the same `type` string and `autoCommit` flag. This is why it is called a **homogeneous** batch: same type for every entry.

The result is a single `RaftReplicationResult`. If the batch succeeds, all entries committed. If it fails, none committed.

10.1.4 When to Use Homogeneous Batches

Use homogeneous batches when you have multiple commands of the same type that do not depend on each other:

- Bulk imports (many `SetValue` commands)
- Batch deletes (many `DeleteValue` commands)
- Event ingestion (many `AppendEvent` commands)

10.2 Batch Replication: Heterogeneous

Sometimes you need to batch commands of different types. A single transaction might set a key, increment a counter, and delete a temporary flag. The homogeneous batch cannot do this because it forces one `type` string across all entries.

`ReplicateEntries` solves this. Each entry carries its own type.

10.2.1 RaftProposalEntry

```
public readonly record struct RaftProposalEntry(  
    string Type,  
    byte[] Data,  
    bool AutoCommit = true,  
    long ExpectedGeneration = 0  
);
```

Each entry has:

- **Type.** The command type string (maps to `RaftLog.LogType` in the state machine).
- **Data.** The serialized command payload (maps to `RaftLog.LogData`).
- **AutoCommit.** Whether this entry commits with the batch. Defaults to `true`.
- **ExpectedGeneration.** A generation fence for partition moves (see Chapter 13). Defaults to `0` (no fence).

10.2.2 ReplicateEntries

```
Task<RaftBatchReplicationResult> ReplicateEntries(  
    int partitionId,  
    IReadOnlyList<RaftProposalEntry> entries,  
    CancellationToken cancellationToken = default  
);
```

This method sends a heterogeneous batch: one proposal, one round-trip, one flush, but each entry has its own type. The result is a `RaftBatchReplicationResult` that contains a per-entry outcome.

10.2.3 Example: Mixed Command Batch

```
var entries = new List<RaftProposalEntry>  
{  
    new(  
        Type: "SetValue",  
        Data: JsonSerializer.SerializeToUtf8Bytes(  
            new SetValueCommand("user:123:name", "Alice")  
        )  
    ),  
    new(  
        Type: "IncrementCounter",  
        Data: JsonSerializer.SerializeToUtf8Bytes(  
            new IncrementCounterCommand("stats:user-updates")  
        )  
    )  
};
```

```

    ),
    new(
        Type: "DeleteValue",
        Data: JsonSerializer.SerializeToUtf8Bytes(
            new DeleteValueCommand("pending:user:123")
        )
    )
);

RaftBatchReplicationResult result = await raft.ReplicateEntries(
    partitionId, entries
);

if (result.Success)
{
    for (int i = 0; i < result.Entries.Count; i++)
    {
        RaftEntryResult entry = result.Entries[i];
        Console.WriteLine(
            $"Entry {i}: status={entry.Status}, logIndex={entry.LogIndex}"
        );
    }
}

```

All three commands travel in one proposal. One quorum round-trip commits the key update, the counter increment, and the delete.

10.2.4 RaftBatchReplicationResult

```

public sealed class RaftBatchReplicationResult
{
    public bool Success { get; }
    public RaftOperationStatus Status { get; }
    public HLCTimestamp TicketId { get; }
    public IReadOnlyList<RaftEntryResult> Entries { get; }
}

```

- **Success.** true if at least one entry was admitted and appended. false on a batch-level rejection (bad shape or every entry fenced out).
- **Status.** The overall batch status.
- **TicketId.** The ticket for the batch proposal. Use this for diagnostics or for manual commit (see below).
- **Entries.** One RaftEntryResult per input entry, aligned by index.

10.2.5 RaftEntryResult

```

public readonly record struct RaftEntryResult(
    RaftOperationStatus Status,

```

```

    long LogIndex,
    HLCTimestamp Ticket
);

```

- **Status.** The outcome for this entry. **Success** for auto-commit entries that committed. **Pending** for manual entries that await explicit commit.
- **LogIndex.** The log index assigned to this entry. -1 if the entry was not appended.
- **Ticket.** The manual-commit ticket for manual entries. **HLCTimestamp.Zero** for auto-commit entries.

10.2.6 Batch Shape Rules

A heterogeneous batch has a shape rule for the order of auto-commit and manual entries:

1. Auto-commit entries (**AutoCommit = true**) must come first.
2. Manual entries (**AutoCommit = false**) must come after all auto-commit entries.
3. No auto-commit entry may follow a manual entry.

This rule exists because manual entries form one contiguous suffix that the application commits or rolls back as a unit. An auto-commit entry after a manual entry would break that property.

Valid batch:

[AutoCommit]	[AutoCommit]	[Manual]	[Manual]
auto group (commits)		manual group (pending)	

Invalid batch:

[AutoCommit]	[Manual]	[AutoCommit]	← rejected
		▲	
		batch-level rejection	

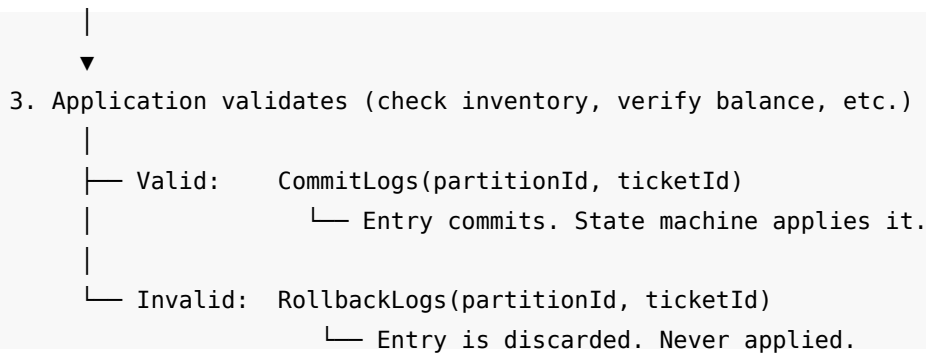
If a batch violates this rule, **ReplicateEntries** returns a batch-level failure. No entries are appended.

10.3 Manual Commit

By default, every entry commits as soon as it reaches a quorum. The **autoCommit** parameter controls this. When **autoCommit** is **false**, the entry is proposed and replicated but not committed. The application decides later whether to commit or roll back.

10.3.1 The Flow

1. Propose with **autoCommit: false**
 |
 ▼
2. Entry replicates to quorum (durable, but not committed)



10.3.2 Step 1: Propose Without Committing

```

byte[] payload = JsonSerializer.SerializeToUtf8Bytes(
    new ReserveInventoryCommand("SKU-42", quantity: 5)
);

RaftReplicationResult result = await raft.ReplicateLogs(
    partitionId,
    "ReserveInventory",
    payload,
    autoCommit: false,
    cancellationToken: timeout.Token
);

if (!result.Success)
{
    // Proposal failed. Handle the error.
    return Results.Json(
        new { error = result.Status.ToString() },
        statusCode: 503
    );
}

// The entry is proposed and durable, but NOT committed.
// result.TicketId identifies this proposal.
HLCTimestamp ticketId = result.TicketId;
  
```

The `TicketId` on the result is an `HLCTimestamp` value. It identifies the proposal for later commit or rollback.

10.3.3 Step 2: Validate

The application runs its validation logic. This might check inventory levels, verify account balances, or apply business rules. The validation reads from the current committed state.

```

int available = GetAvailableInventory("SKU-42");
bool valid = available >= 5;
  
```

10.3.4 Step 3: Commit or Roll Back

```
if (valid)
{
    var (success, status, commitLogId) = await raft.CommitLogs(
        partitionId,
        ticketId,
        cancellationToken: timeout.Token
    );

    if (success)
    {
        return Results.Ok(new { reserved = true, commitLogId });
    }

    return Results.Json(
        new { error = status.ToString() },
        statusCode: 503
    );
}
else
{
    var (success, status, _) = await raft.RollbackLogs(
        partitionId,
        ticketId,
        cancellationToken: timeout.Token
    );

    return Results.Json(
        new { reserved = false, reason = "Insufficient inventory" },
        statusCode: 409
    );
}
```

10.3.5 CommitLogs

```
Task<(bool success, RaftOperationStatus status, long commitLogId)> CommitLogs(
    int partitionId,
    HLCTimestamp ticketId,
    CancellationToken cancellationToken = default
);
```

CommitLogs commits the entries associated with the given ticket. After the call succeeds, the entries are committed and the state machine applies them through OnReplicationReceived.

The return tuple contains:

- **success.** true if the commit succeeded.

- **status.** The operation status.
- **commitLogId.** The log id of the commit record.

Committing an already-committed ticket is a no-op. This makes retries safe.

10.3.6 RollbackLogs

```
Task<(bool success, RaftOperationStatus status, long commitLogId)> RollbackLogs(
    int partitionId,
    HLCTimestamp ticketId,
    CancellationToken cancellationToken = default
);
```

RollbackLogs discards the entries associated with the given ticket. The entries are removed from the log. The state machine never sees them.

Rolling back an already-rolled-back ticket is also a no-op.

10.3.7 Cancellation Behavior

Both CommitLogs and RollbackLogs handle cancellation the same way. If the CancellationToken fires before the operation completes, the method returns (false, OperationCancelled, 0) instead of throwing. The caller should back off and retry the same ticket. The queued work may still apply, and re-issuing the same ticket is safe.

10.4 The Reserve-and-Confirm Pattern

Here is a complete example that uses manual commit for an inventory reservation:

```
record ReserveInventoryCommand(string Sku, int Quantity, string OperationId);
record ConfirmReservationCommand(string Sku, int Quantity, string OperationId);

app.MapPost("/reserve", async (HttpRequest request) =>
{
    using var reader = new StreamReader(request.Body);
    string body = await reader.ReadToEndAsync();
    var input = JsonSerializer.Deserialize<ReserveRequest>(body);

    if (input is null)
        return Results.BadRequest();

    string operationId = Guid.NewGuid().ToString();

    // Step 1: Propose the reservation without committing
    byte[] payload = JsonSerializer.SerializeToUtf8Bytes(
        new ReserveInventoryCommand(input.Sku, input.Quantity, operationId)
    );

    using var timeout = new CancellationTokenSource(
        TimeSpan.FromSeconds(5)
    );
```

```

var result = await raft.ReplicateLogs(
    partitionId,
    "ReserveInventory",
    payload,
    autoCommit: false,
    cancellationToken: timeout.Token
);

if (!result.Success)
{
    return Results.Json(
        new { error = result.Status.ToString() },
        statusCode: 503
    );
}

HLCTimestamp ticketId = result.TicketId;

// Step 2: Validate against current committed state
int available = GetAvailableInventory(input.Sku);

if (available < input.Quantity)
{
    // Not enough inventory. Roll back.
    await raft.RollbackLogs(partitionId, ticketId, timeout.Token);

    return Results.Json(
        new { reserved = false, reason = "Insufficient inventory" },
        statusCode: 409
    );
}

// Step 3: Commit the reservation
var (success, status, commitLogId) = await raft.CommitLogs(
    partitionId,
    ticketId,
    timeout.Token
);

if (!success)
{
    return Results.Json(
        new { error = status.ToString() },

```



```

        statusCode: 503
    );
}

return Results.Ok(new
{
    reserved = true,
    sku = input.Sku,
    quantity = input.Quantity,
    commitLogId
});
});

record ReserveRequest(string Sku, int Quantity);

```

This pattern gives the application a decision point between proposal and commit. The entry is durable on a quorum of nodes, but the state machine does not apply it until the explicit commit.

10.5 Manual Commit with Heterogeneous Batches

`ReplicateEntries` supports mixing auto-commit and manual entries in one batch. The auto-commit entries commit immediately. The manual entries form a trailing group that the application commits or rolls back later.

```

var entries = new List<RaftProposalEntry>
{
    // Auto-commit: record the attempt (always commits)
    new(
        Type: "AuditLog",
        Data: JsonSerializer.SerializeToUtf8Bytes(
            new AuditLogEntry("reservation-attempt", sku, quantity)
        ),
        AutoCommit: true
    ),
    // Manual: the reservation itself (commit or rollback)
    new(
        Type: "ReserveInventory",
        Data: JsonSerializer.SerializeToUtf8Bytes(
            new ReserveInventoryCommand(sku, quantity, operationId)
        ),
        AutoCommit: false
    )
};

RaftBatchReplicationResult result = await raft.ReplicateEntries(
    partitionId, entries

```

```

);

if (result.Success)
{
    // The audit log entry committed immediately (status = Success).
    // The reservation entry is pending (status = Pending).
    RaftEntryResult auditResult = result.Entries[0];
    RaftEntryResult reserveResult = result.Entries[1];

    // The manual ticket for commit/rollback
    HLCTimestamp ticketId = reserveResult.Ticket;

    // Validate, then commit or rollback ticketId...
}

```

The audit log entry commits with the batch. The reservation entry waits for an explicit `CommitLogs` or `RollbackLogs`.

10.6 What Happens When the Leader Crashes

If the leader crashes after a manual proposal but before the application calls `CommitLogs`:

1. The entry is durable on a quorum of nodes (it was proposed and acknowledged).
2. The new leader does not automatically commit the entry. It remains uncommitted.
3. The client's `CommitLogs` call fails (the connection breaks or times out).
4. The entry stays in the log as an uncommitted proposal until a new leader either includes it in its own term or overwrites it.

The client must detect the failure and re-propose the entire operation:

```

var (success, status, _) = await raft.CommitLogs(
    partitionId, ticketId, timeout.Token
);

if (!success)
{
    if (status == RaftOperationStatus.OperationCancelled ||
        status == RaftOperationStatus.NodeIsNotLeader)
    {
        // Leader changed. Re-propose from scratch.
        // The old uncommitted entry will be overwritten
        // by the new leader.
    }
}

```

This is why manual commit is a more advanced pattern. It adds a failure window between proposal and commit where a leader crash requires re-proposal. Auto-commit does not have this window because the entry commits as part of the same proposal.

10.7 Checkpoints

`ReplicateCheckpoint` writes a special marker entry to the log. This marker tells the WAL compaction process that all entries before this point are safe to truncate.

```
RaftReplicationResult result = await raft.ReplicateCheckpoint(  
    partitionId,  
    cancellationToken: timeout.Token  
);
```

The checkpoint is a replicated entry, not a local-only marker. It goes through the same proposal and quorum process as any other entry. This ensures that all nodes agree on the checkpoint position.

10.7.1 When to Use Checkpoints

Use checkpoints when:

- The WAL grows large and you want to allow compaction.
- Your application has a natural “save point” (e.g., after a snapshot of the state machine).
- You want to bound the replay time on restart.

Without checkpoints, the WAL retains every entry from the beginning. Each restart replays the entire log through `OnLogRestored`. A checkpoint lets the WAL discard entries before the checkpoint, so replay starts from the checkpoint position instead of the beginning.

10.7.2 Checkpoint Flow

```
Log:  [1] [2] [3] [4] [5] [CP] [6] [7] [8]  
                                     ▲  
                                   checkpoint at index 5  
  
After compaction:  
  
Log:  [CP] [6] [7] [8]  
  
On restart, replay starts at [6], not [1].
```

The checkpoint does not include a snapshot of the state machine. It is a marker only. The application must ensure that the state machine can reconstruct its state from the entries after the checkpoint. This typically means the application writes a snapshot to disk at the same time it issues the checkpoint.

10.8 Building a Bulk Import Endpoint

Here is a practical endpoint that uses batch replication:

```
app.MapPost("/store/bulk", async (HttpRequest request) =>  
{  
    bool isLeader = await raft.AmILeader(partitionId, default);
```

```

if (!isLeader)
{
    string? leader = raft.GetPartitionLeaderHint(partitionId);
    return Results.Json(
        new { error = "NotLeader", leader },
        statusCode: 503
    );
}

using var reader = new StreamReader(request.Body);
string body = await reader.ReadToEndAsync();
var items = JsonSerializer.Deserialize<List<KeyValuePair>>(body);

if (items is null || items.Count == 0)
    return Results.BadRequest();

// Build the batch
var entries = items.Select(item => new RaftProposalEntry(
    Type: "SetValue",
    Data: JsonSerializer.SerializeToUtf8Bytes(
        new SetValueCommand(item.Key, item.Value)
    )
)).ToList();

using var timeout = new CancellationTokenSource(
    TimeSpan.FromSeconds(10)
);

RaftBatchReplicationResult result = await raft.ReplicateEntries(
    partitionId, entries, timeout.Token
);

if (result.Success)
{
    int committed = result.Entries.Count(
        e => e.Status == RaftOperationStatus.Success
    );

    return Results.Ok(new { imported = committed, total = items.Count });
}

return Results.Json(
    new { error = result.Status.ToString() },
    statusCode: 503
);

```

```
);
});

record KeyValueItem(string Key, string Value);
```

This endpoint accepts a JSON array of key-value pairs and imports all of them in one proposal. The response reports how many entries committed.

10.8.1 Performance Comparison

Approach	100 items, 1 ms RTT	1000 items, 1 ms RTT
Single-entry loop	~100 ms	~1000 ms
Homogeneous batch	~1 ms	~1 ms
Heterogeneous batch	~1 ms	~1 ms

The batch approaches reduce the time from $O(N)$ round-trips to $O(1)$. The WAL also benefits: one flush for the entire batch instead of one flush per entry.

10.9 Choosing the Right Approach

Need	Use
Many commands, same type	<code>ReplicateLogs</code> with <code>IReadOnlyList<byte[]></code>
Many commands, different types	<code>ReplicateEntries</code> with <code>RaftProposalEntry</code> list
Validate before committing	<code>autoCommit: false</code> + <code>CommitLogs</code> / <code>RollbackLogs</code>
Mark a WAL compaction point	<code>ReplicateCheckpoint</code>

Start with auto-commit. Move to manual commit only when you need the validate-then-decide pattern. Manual commit adds complexity and a failure window between proposal and commit.

10.10 Summary

- Batch replication sends multiple entries in one proposal. One quorum round-trip and one WAL flush replace N round-trips and N flushes.
- Homogeneous batches (`ReplicateLogs` with a list) require all entries to share the same type.
- Heterogeneous batches (`ReplicateEntries`) allow each entry to carry its own type and auto-commit flag.
- `RaftBatchReplicationResult` contains per-entry results aligned by index to the input list.
- Manual commit (`autoCommit: false`) proposes entries without committing. The application calls `CommitLogs` to commit or `RollbackLogs` to discard.
- The `TicketId` (an `HLCTimestamp`) connects a proposal to its commit or rollback.

- If the leader crashes before a manual commit, the entry is not automatically committed. The client must re-propose.
- `ReplicateCheckpoint` marks a point in the log for WAL compaction. Entries before the checkpoint can be truncated.
- The next chapter explains why a single Raft group is not sufficient for large-scale systems and introduces partitioned Raft.

11 Why One Raft Group Is Not Enough

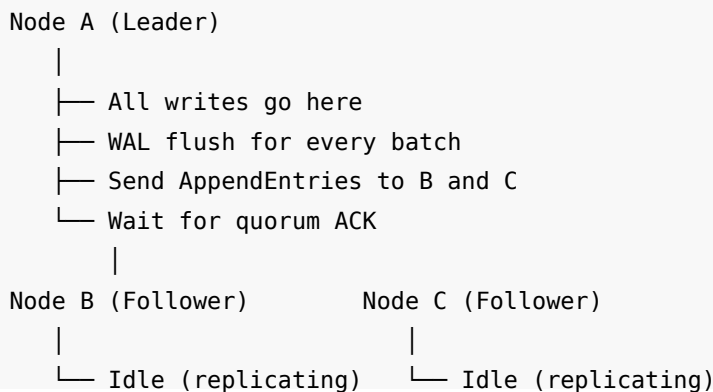
Every chapter so far used a single partition: partition 1. All writes went to one leader. All reads came from one state machine. This works for small workloads. It does not scale.

This chapter explains why and introduces partitioned Raft: independent Raft groups with independent leaders that distribute the write load across all nodes in the cluster.

11.1 The Single-Leader Bottleneck

In a Raft cluster with one partition, only the leader accepts writes. The followers replicate entries and serve as standby copies. They do not accept proposals.

Three-node cluster, one partition:



Node A does all the work. Nodes B and C receive entries and store them, but they do not propose or order commands. The cluster has the capacity of three nodes, but the write throughput is limited to what one node can handle.

11.1.1 Measuring the Bottleneck

Assume each node can handle 50,000 writes per second. With one partition:

Metric	Value
Write throughput	50,000 ops/sec (one node)
Leader CPU usage	High
Follower CPU usage	Low
Cluster utilization	33% (1 of 3 nodes busy)

Two-thirds of the cluster’s capacity sits idle. Adding more nodes does not help. A six-node cluster with one partition still funnels all writes through one leader. The write throughput stays at 50,000 ops/sec.

11.1.2 The Read Side

Reads are less affected. Every node has a copy of the state. A read that tolerates eventual consistency can go to any node (Chapter 6). A consistent read still needs a quorum confirmation, but the confirmation overhead is small compared to a write.

The bottleneck is writes. Raft is a write-path protocol. Its core job is to order and replicate write proposals. When all proposals go through one leader, that leader is the ceiling.

11.2 Partitioned Raft

The solution: run multiple independent Raft groups in the same cluster. Each group is a **partition**. Each partition has its own leader, its own term, its own log, and its own state machine.

Three-node cluster, three partitions:

Node A	Node B	Node C
P1 LEADER	P1 follow	P1 follow
P2 follow	P2 LEADER	P2 follow
P3 follow	P3 follow	P3 LEADER

Writes to partition 1 → Node A

Writes to partition 2 → Node B

Writes to partition 3 → Node C

Now each node leads one partition. Write load spreads across all three nodes. The cluster throughput scales with the number of nodes and partitions.

Metric	One partition	Three partitions
Write throughput	50,000 ops/sec	150,000 ops/sec
Leader CPU usage	1 node at high	3 nodes at moderate
Cluster utilization	33%	~100%

11.2.1 Independence

Each partition is fully independent. The leader of partition 1 does not coordinate with the leader of partition 2. Their logs are separate. Their terms are separate. Their elections are separate.

If partition 2’s leader crashes, partition 2 runs a new election. Partitions 1 and 3 continue without interruption. The failure scope is one partition, not the entire cluster.

Node B crashes:

Node A

P1 LEADER	(unaffected)
P2 follow	← election starts →
P3 follow	(unaffected)

Node C

P1 follow	
P2 LEADER	(new leader)
P3 LEADER	

Partition 1: no interruption

Partition 2: brief pause, then resumes on Node C

Partition 3: no interruption

11.3 Configuring Partitions

The `InitialPartitions` property on `RaftConfiguration` sets the number of application partitions at cluster bootstrap:

```
RaftConfiguration configuration = new()
{
    NodeName = "node-a",
    Host = "localhost",
    Port = 8001,
    InitialPartitions = 8
};
```

This creates 8 application partitions (partition 1 through partition 8) plus the system partition (partition 0). The system partition is always present. The `InitialPartitions` count does not include it.

11.3.1 Choosing the Number of Partitions

The number of partitions determines the maximum parallelism for writes:

Partitions	Maximum leaders	Write parallelism
1	1	Single node
3	3	All 3 nodes (on a 3-node cluster)
8	3	All 3 nodes, some lead multiple partitions
24	3	All 3 nodes, 8 partitions each

More partitions than nodes is fine. Kommander distributes leadership across the available nodes. With 8 partitions on 3 nodes, one node leads approximately 3 partitions and the other two lead approximately 2 or 3 each.

Guidelines:

- **Minimum:** At least as many partitions as nodes. This ensures every node can lead at least one partition.

- **Practical default:** 2 to 4 times the number of nodes. This gives the scheduler room to balance leadership even when a node goes down.
- **Too many:** Each partition has overhead (WAL files, memory for the state machine, election timers). Hundreds of partitions on a 3-node cluster waste resources for no throughput gain.

11.3.2 InitialPartitions Is a Bootstrap Setting

`InitialPartitions` takes effect only during the first cluster bootstrap (the first leader election on partition 0). The value is written into the committed partition map. On subsequent restarts, the value in the configuration is ignored. The cluster uses the committed map.

To add partitions to a running cluster, use the partition management APIs (Chapter 13). To remove partitions, use the partition lifecycle APIs. `InitialPartitions` does not change the partition count of an existing cluster.

11.4 Partition 0: The System Partition

Partition 0 is the **system partition**. Kommander reserves it for internal cluster configuration: the partition map, membership changes, and other system-level operations.

```
Partition 0 (system):  cluster configuration
Partition 1 (application): your data
Partition 2 (application): your data
Partition 3 (application): your data
...
```

Rules for partition 0:

- Do not replicate application data to partition 0.
- Do not register application callbacks on partition 0.
- Kommander manages partition 0 internally.
- System entries use the `_RaftSystem` log type.

Your application works with partitions 1 and above. When the previous chapters used `int partitionId = 1;`, that was partition 1, the first application partition.

11.5 Routing Keys to Partitions

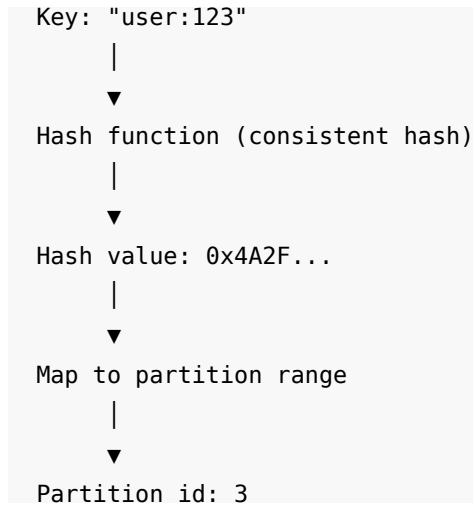
With multiple partitions, the application must decide which partition holds which data. Kommander provides `GetPartitionKey` for this:

```
int partitionId = raft.GetPartitionKey("user:123");
```

`GetPartitionKey` hashes the input string and maps the hash to a partition id. The same key always maps to the same partition. Different keys may map to different partitions.

The method returns a partition id in the application range (1 and above). It never returns 0 (the system partition).

11.5.1 How the Routing Works



The partition map defines ranges. Each partition owns a range of hash values. `GetPartitionKey` hashes the key, finds which range contains the hash, and returns that partition's id.

11.5.2 Prefix Routing

`GetPrefixPartitionKey` works the same way but uses only a prefix of the key for routing:

```
int partitionId = raft.GetPrefixPartitionKey("user:123:profile");
```

This is useful when you want all keys with the same prefix to land on the same partition. For example, all keys starting with "user:123" go to the same partition, regardless of the suffix.

11.5.3 Updated ReplicatedStore

Here is the `ReplicatedStore` evolved from one partition to multiple partitions:

```
// Configuration: 8 partitions across 3 nodes
RaftConfiguration configuration = new()
{
    NodeName = args[0],
    Host = "localhost",
    Port = int.Parse(args[1]),
    InitialPartitions = 8
};

// Per-partition state machines
var stores = new ConcurrentDictionary<int, ConcurrentDictionary<string,
string>>();

ConcurrentDictionary<string, string> GetStore(int partitionId)
{
    return stores.GetOrAdd(partitionId, _ => new ConcurrentDictionary<string,
```

```

string>());
}

Task<bool> ApplyCommand(int partitionId, RaftLog log)
{
    var store = GetStore(partitionId);

    switch (log.LogType)
    {
        case "SetValue":
            var cmd = JsonSerializer.Deserialize<SetValueCommand>(
                log.LogData ?? []
            );
            if (cmd is not null)
                store[cmd.Key] = cmd.Value;
            break;

        case "DeleteValue":
            var del = JsonSerializer.Deserialize<DeleteValueCommand>(
                log.LogData ?? []
            );
            if (del is not null)
                store.TryRemove(del.Key, out _);
            break;
    }

    return Task.FromResult(true);
}

raft.OnReplicationReceived += ApplyCommand;
raft.OnLogRestored += ApplyCommand;

```

The state machine now receives the `partitionId` parameter and maintains a separate dictionary for each partition. This is important: each partition is an independent Raft group with its own log. The state machine must keep the data separate.

11.5.4 Routing Writes

```

app.MapPut("/store/{key}", async (string key, HttpRequest request) =>
{
    // Route the key to a partition
    int partitionId = raft.GetPartitionKey(key);

    // Check if this node leads the partition
    bool isLeader = await raft.AmILeader(partitionId, default);
    if (!isLeader)

```

```

{
    string? leader = raft.GetPartitionLeaderHint(partitionId);
    return Results.Json(
        new { error = "NotLeader", leader, partitionId },
        statusCode: 503
    );
}

using var reader = new StreamReader(request.Body);
string value = await reader.ReadToEndAsync();

byte[] payload = JsonSerializer.SerializeToUtf8Bytes(
    new SetValueCommand(key, value)
);

using var timeout = new CancellationTokenSource(
    TimeSpan.FromSeconds(5)
);

var result = await raft.ReplicateLogs(
    partitionId, "SetValue", payload,
    cancellation_token: timeout.Token
);

if (result.Success)
    return Results.Ok(new { key, value, partitionId });

string? newLeader = raft.GetPartitionLeaderHint(partitionId);
return Results.Json(
    new { error = result.Status.ToString(), leader = newLeader },
    statusCode: 503
);
});

```

The key change: `partitionId` is no longer hardcoded to 1. It comes from `GetPartitionKey(key)`. Different keys route to different partitions. Different partitions may have different leaders. The error response now includes `partitionId` so the client knows which partition to retry.

11.5.5 Routing Reads

```

app.MapGet("/store/{key}", async (string key, HttpRequest request) =>
{
    int partitionId = raft.GetPartitionKey(key);
    var store = GetStore(partitionId);

```

```

bool consistent = request.Query.ContainsKey("consistent");

if (consistent)
{
    using var timeout = new CancellationTokenSource(
        TimeSpan.FromSeconds(3)
    );

    bool confirmed = await raft.ConfirmLeadershipAsync(
        partitionId, timeout.Token
    );

    if (!confirmed)
    {
        string? leader = raft.GetPartitionLeaderHint(partitionId);
        return Results.Json(
            new { error = "LeadershipNotConfirmed", leader, partitionId },
            statusCode: 503
        );
    }
}

return store.TryGetValue(key, out string? value)
    ? Results.Ok(new { key, value, partitionId })
    : Results.NotFound();
});

```

The read handler uses the same routing. Consistent reads confirm leadership on the correct partition.

11.6 The Partition Map

GetPartitionMap returns the current partition map:

```

IReadOnlyList<RaftPartitionRange> map = raft.GetPartitionMap();

```

```

foreach (RaftPartitionRange range in map)
{
    Console.WriteLine(
        $"Partition {range.PartitionId}: " +
        $"range [{range.StartRange}, {range.EndRange}], " +
        $"generation {range.Generation}, " +
        $"state {range.State}"
    );
}

```

Each RaftPartitionRange has:

Property	Type	Description
PartitionId	int	The partition's id
StartRange	int	Start of the hash range
EndRange	int	End of the hash range
Generation	long	Incremented on each map mutation
State	RaftPartitionState	Lifecycle state (active, splitting, etc.)
RoutingMode	RaftRoutingMode	Whether included in GetPartitionKey routing
Replicas	List<RaftReplica>	The nodes hosting this partition

The partition map is replicated through partition 0 (the system partition). All nodes see the same map.

11.7 Failure Isolation

With multiple partitions, a single node failure affects only the partitions that node led. Other partitions continue without interruption.

11.7.1 Example: Node B Fails

Before failure:

Node A: leads P1, P4, P7

Node B: leads P2, P5, P8

Node C: leads P3, P6

Node B crashes.

After election:

Node A: leads P1, P4, P7, P5 (new)

Node C: leads P3, P6, P2 (new), P8 (new)

Affected partitions: P2, P5, P8 (brief pause during election)

Unaffected partitions: P1, P3, P4, P6, P7

Clients that wrote to partitions P1, P3, P4, P6, or P7 never noticed the failure. Clients that wrote to P2, P5, or P8 saw a brief timeout (the election timeout, 2 to 4 seconds), then resumed on the new leaders.

This is the core advantage of partitioned Raft: **failure isolation**. One node failure does not stop the entire cluster. It stops only the partitions that node led, and those partitions recover through election within seconds.

11.8 A Note on the Word “Partition”

The word “partition” appears in three contexts in distributed systems:

1. **Data partition.** A subset of the data, assigned to a Raft group. This is the meaning in this chapter.
2. **Raft group / partition.** The Raft consensus group that manages a data partition. In Kommander, these are the same thing: one Raft group per data partition.
3. **Network partition.** A failure that splits the network into isolated groups (Chapter 9).

Context makes the meaning clear. When this book uses “partition” without a qualifier, it means a Raft group (senses 1 and 2). “Network partition” always has the “network” qualifier.

11.9 Summary

- One Raft group means one leader. One leader is a write throughput bottleneck.
- Partitioned Raft runs multiple independent Raft groups in the same cluster. Each partition has its own leader, term, log, and state machine.
- `InitialPartitions` sets the number of application partitions at bootstrap. Partition 0 is reserved for system use.
- `GetPartitionKey` hashes a string key to a partition id. The same key always maps to the same partition.
- Each partition must have its own state machine (separate data store). The `partitionId` parameter in the callbacks identifies which partition the entry belongs to.
- A node failure affects only the partitions that node led. Other partitions continue without interruption.
- More partitions allow better distribution of leadership across nodes. A practical default is 2 to 4 times the node count.
- The next chapter builds a fully partitioned `ReplicatedStore` with per-partition routing and state machines.